

Universidade Federal de Viçosa
Campus Rio Paranaíba

Victor Felipe de Oliveira - 8170

Análise de Complexidade de Algoritmos de Ordenação e Busca

Rio Paranaíba - MG

2025

Universidade Federal de Viçosa
Campus **Rio Paranaíba**

Victor Felipe de Oliveira - 8170

Análise de Complexidade de Algoritmos de Ordenação e Busca

Trabalho apresentado para obtenção de créditos na disciplina SIN213 - Projeto de Algoritmo da Universidade Federal de Viçosa - Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba - MG
2025

1 RESUMO

Os algoritmos constituem elementos centrais da ciência da computação, representando sequências estruturadas de instruções que transformam problemas complexos em soluções precisas e automatizadas. Eles permeiam praticamente todas as tecnologias contemporâneas, desde sistemas de busca e recomendação até softwares de gestão, inteligência artificial e análise de grandes volumes de dados. Compreender a lógica, a organização e a aplicabilidade de diferentes algoritmos permite desenvolver uma visão crítica sobre eficiência, escalabilidade e adaptabilidade de sistemas computacionais, reforçando a importância do estudo algorítmico para a formação acadêmica e profissional, bem como para a capacitação em resolução estruturada de problemas.

A análise da complexidade de algoritmos constitui um aspecto central do estudo de algoritmos, permitindo avaliar quantitativamente o desempenho de soluções em função do tamanho da entrada. A complexidade temporal refere-se ao tempo de execução necessário para processar dados, enquanto a complexidade espacial relaciona-se à quantidade de memória requerida. Compreender essas métricas é essencial para identificar limites, potencialidades e escalabilidade de algoritmos em diferentes contextos.

Além da análise teórica, a avaliação de complexidade inclui conceitos como melhor, pior e caso médio, bem como notações formais como Big-O, Big-Theta (Θ) e Big-Omega (Ω), que fornecem uma visão rigorosa sobre o comportamento de algoritmos em diversas situações. O estudo sistemático dessas propriedades permite que estudantes e profissionais desenvolvam habilidades críticas para selecionar, adaptar e otimizar soluções computacionais, considerando tanto restrições de tempo quanto de memória.

No cotidiano, os algoritmos podem parecer ser invisíveis, porém impactam diretamente diversas áreas e sistemas tecnológicos. Motores de busca, redes sociais, sistemas de navegação, ferramentas de análise científica, entre muitos outros, dependem da aplicação de algoritmos eficientes para oferecer desempenho confiável e escalabilidade. Avaliar a complexidade desses algoritmos permite compreender o impacto de diferentes escolhas de implementação e

otimização sobre o funcionamento de sistemas em larga escala.

Este relatório analisa algoritmos selecionados na literatura e suas particularidades, com ênfase em suas complexidades temporais e espaciais, essenciais para avaliar desempenho e consumo de recursos conforme o tamanho da entrada cresce.

Sumário

1	RESUMO	1
2	INTRODUÇÃO	5
3	ALGORITMOS	8
3.1	Insertion Sort	8
3.2	Bubble Sort	10
3.3	Selection Sort	12
3.4	Shell Sort	13
3.5	Merge Sort	15
3.6	Quick Sort	17
3.7	Heap Sort	20
4	ANÁLISE DE COMPLEXIDADE	22
4.1	Insertion Sort	22
4.2	Bubble Sort	26
4.3	Selection Sort	30
4.4	Shell Sort	33
4.5	Merge Sort	36
4.6	Quick Sort	39

4.7	Heap Sort	43
5	TABELA E GRÁFICO	49
5.1	Insertion Sort	49
5.2	Bubble Sort	50
5.3	Selection Sort	51
5.4	Shell Sort	52
5.5	Comparação dos algoritmos	53
5.6	Merge Sort	56
5.7	Quick Sort	57
5.8	Heap Sort	60
6	CONCLUSÃO	61
7	REFERÊNCIAS BIBLIOGRÁFICAS	63

2 INTRODUÇÃO

Algoritmos são a base de praticamente toda a computação moderna e da ciência da computação, e embora hoje sejam frequentemente associados a programação, sua história antecede a invenção das máquinas digitais. Um algoritmo pode ser definido, de forma geral, como um conjunto de passos bem definidos que levam à solução de um problema (Cormen et al., 2013). A ideia de procedimento bem definido e sistemático para resolver tarefas existe desde civilizações antigas e vêm evoluindo com o passar do tempo, tornando-se uma das bases fundamentais da tecnologia e do pensamento lógico.

A origem da palavra "algoritmo" remonta ao matemático persa Muhammad ibn Musa al-Khwarizmi, famoso por seus tratados sobre aritmética e álgebra, o nome "algoritmo" deriva da forma como seu nome foi traduzida para o latim, *Algoritmi* (Knuth, 1997). Seus trabalhos apresentavam modelos detalhados para realizar operações matemáticas, como somas e multiplicações, permitindo cálculos mais eficientes e acessíveis, estabelecendo as bases do raciocínio algorítmico.

Além de sua secular história, os algoritmos possuem importância prática e teórica imensa. São eles que possibilitam que computadores resolvam problemas complexos de forma automatizada. Cada software que utilizamos é, em sua essência, um conjunto de algoritmos cuidadosamente elaborados para realizar tarefas específicas, podendo se estender a renderizar uma página da web até controlar gerenciar recursos de um sistema operacional (Cormen et al., 2013). Algoritmos eficientes permitem resolver problemas de forma mais rápida e objetiva, com menor consumo de recursos, o que é essencial para o bom funcionamento dos computadores, que possuem recursos limitados e onde a quantidade de informações processadas a cada segundo é gigantesca.

O estudo da complexidade de algoritmos é um campo fundamental dentro da ciência da computação teórica. Ele busca analisar e classificar algoritmos de acordo com o tempo e a quantidade de memória que consomem conforme o tamanho da entrada cresce (Sipser, 2012). Essa análise permite prever o desempenho de um algoritmo e compará-lo a outros, possibi-

litando ter um maior conhecimento sobre qual melhor algoritmo para determinada situação. A notação O (Big O) é amplamente utilizada para expressar a ordem de crescimento do tempo de execução, Com $O(n)$, $O(n^2)$, $O(\log n)$ e assim por diante (Cormen et al., 2013). Analisar e compreender a complexidades dos algoritmos é fundamental porque, mesmo que dois algoritmos resolvam o mesmo problema, um pode ter um desempenho drasticamente superior ao outro, principalmente ao se tratar de quantidades de dados maiores. Um algoritmo com tempo exponencial, por exemplo, pode se tornar inviável a medida que a entrada vai crescendo, enquanto um com tempo polinomial pode resolvê-las de forma prática.

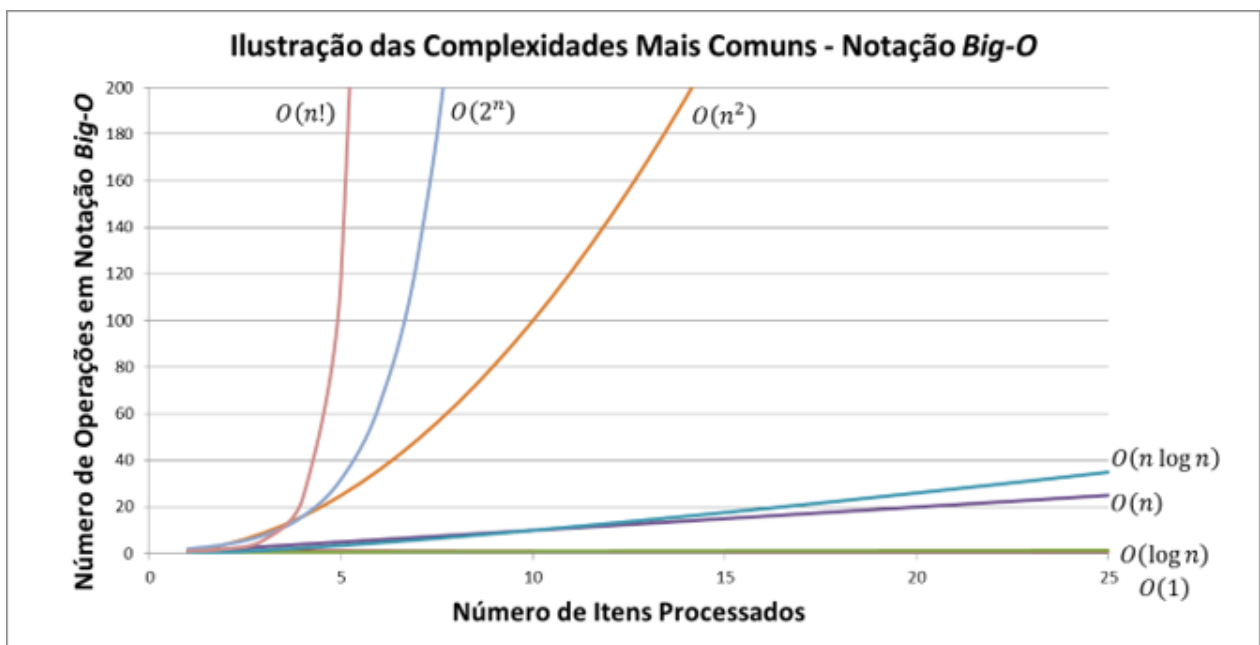


Figura 1: Gráfico de custo de operação dos algoritmos baseado na notação Big O

Indo além da teoria, os algoritmos estão profundamente relacionados ao nosso cotidiano, de forma imperceptível muitas das vezes. Seja em motores de busca da internet que utilizam de algoritmos de indexação e ranqueamento para apresentar os resultados mais relevantes em milésimos de segundos, redes sociais que utilizam de algoritmos de recomendação para selecionar quais conteúdos serão exibidos aos seus usuários com base em seus interesses e comportamentos, operações bancárias, diagnósticos médicos, até mesmo grandes modelos de redes neurais dependem de algoritmos eficientes para funcionar de maneira rápida e confiável.

Este relatório busca compreender os conceitos e técnicas por trás dos principais algoritmos

de ordenação e busca existentes e suas respectivas complexidades, para a formação de uma base sólida de conhecimento e para a tomada de decisões mais assertivas no desenvolvimento de sistemas computacionais. O estudo comparativo dessas técnicas, junto com a análise de suas complexidades computacionais, torna-se indispensável para a construção de soluções que atendam às crescentes demandas por desempenho e escalabilidade na área da tecnologia da informação.

3 ALGORITMOS

3.1 Insertion Sort

O Algoritmo de ordenação por inserção, ou simplesmente *Insertion Sort*, é um dos algoritmos mais antigos, simples e didáticos dentro dos campos de estudos da ciência da computação. Sua lógica se baseia em um conceito bastante intuitivo: construir uma lista ordenada gradativamente a partir de um conjunto desordenado, ordenando um elemento por vez. Em termos práticos, o algoritmo mantém uma parte inicial da lista já organizada e, a cada passo, retira um item da outra parte da lista que está desordenada, realocando-o no ponto exato dentro da sequência. O processo é repetido até que todos os elementos estejam alinhados na ordem desejada.

Um exemplo clássico que ilustra seu funcionamento é o de um jogador arrumando cartas de um baralho em sua mão. Ao receber a primeira carta, ela já está em ordem por estar sozinha. Assim que recebe a segunda carta, ela é comparada com a primeira e posicionada à frente ou atrás dela, de acordo com sua ordem de importância dentro do jogo em relação a primeira. Na terceira carta, é necessário encontrar a posição correta entre as outras duas que já estão organizadas. O processo se repete até que todas as cartas na mão do jogador estejam arrumadas. O *Insertion Sort* utiliza da mesma estratégia para organizar listas e vetores numéricos, sempre garantindo que a parte inicial da estrutura esteja organizada enquanto o restante é processado.

Apesar de simples, esse algoritmo apresenta características relevantes. Ele é estável, pois preserva a ordem relativa de elementos iguais, característica vantajosa em aplicações que precisam manter dados correlacionados, além disso, ele utiliza o método *in-place*, que realiza a ordenação utilizando poquíssima memória adicional, utilizando apenas variáveis auxiliares, o tornando muito eficiente em situações cuja capacidade de memória é limitada.

Em questões de desempenho, o *Insertion Sort* se comporta de maneira diferente conforme a entrada que for especificada. No melhor cenário, quando os dados já estão ordenados ou

próximos da ordem final, ele trabalha em tempo linear $O(n)$, já que cada item encontra sua posição quase imediatamente. Porém para os piores e médios casos, o número de comparações e movimentos pode crescer de forma quadrática, levando a uma complexidade de $O(n^2)$. Por estes motivos, embora seja prático para entradas com tamanhos pequenos, não é adequado para grandes volumes de dados, onde outros algoritmos atingem desempenhos menores.

Outro ponto a se destacar é que o Insertion Sort é adaptativo. Isso significa que ele se ajusta ao grau de organização inicial da lista: quanto mais próxima da ordem final estiver a entrada, menos operações serão necessárias. Esse comportamento adaptativo faz com que muitas implementações modernas de algoritmos mais sofisticados recorram ao Insertion Sort para lidar com sublistas pequenas, já que sua simplicidade e baixo custo de execução superam a complexidade de algoritmos recursivos em casos reduzidos.

Na prática, mesmo não sendo o mais rápido para dados grandes, o Insertion Sort tem usos específicos. Ele funciona bem quando os conjuntos são pequenos, quando a lista está quase organizada ou como parte de algoritmos híbridos. Seu baixo consumo de memória, previsibilidade e estabilidade tornam-no útil em sistemas embarcados, em ambientes de recursos limitados e em cenários onde a manutenção da ordem original é um requisito.

3.2 Bubble Sort

O algoritmo *Bubble Sort* é provavelmente o mais conhecido e um dos primeiros a ser ensinado em cursos de algoritmos e estruturas de dados. Sua fama se deve principalmente à simplicidade da sua lógica, ao fácil entendimento do seu funcionamento e à clareza didática com que ilustra conceitos fundamentais de ordenação. O nome “bubble” (bolha) foi atribuído pela forma como os elementos maiores parecem “subir” em direção ao final do vetor, lembrando o movimento de bolhas em um líquido, enquanto os menores vão “descendo” para o início após sucessivas comparações e trocas.

A ideia básica é percorrer o vetor diversas vezes, comparando pares de elementos vizinhos. Sempre que dois elementos estão fora de ordem, eles são trocados. Esse processo é repetido até que nenhuma troca seja necessária, indicando que o vetor já está ordenado. Em cada passagem completa, o maior elemento remanescente é deslocado para a última posição não ordenada.

Um exemplo clássico para compreender o Bubble Sort é imaginar uma fila de pessoas organizadas por altura. Cada pessoa olha para quem está imediatamente à sua frente e troca de lugar caso esteja em posição errada. Depois de uma passagem completa, a pessoa mais alta estará ao fundo da fila. O processo se repete até que todos estejam corretamente posicionados. Essa analogia simples mostra a mecânica de comparações e trocas repetitivas que caracterizam o algoritmo.

Em termos de implementação, o Bubble Sort é considerado um algoritmo *in-place*, pois não exige memória adicional significativa além de algumas variáveis auxiliares. Além disso, ele é estável, garantindo que elementos iguais mantenham sua ordem relativa inicial, algo importante em casos onde atributos secundários precisam ser preservados.

Apesar dessas vantagens conceituais, o Bubble Sort tem sérias limitações práticas. Sua estratégia é considerada ingênua, uma vez que realiza inúmeras comparações desnecessárias, mesmo em listas que já estão parcialmente ordenadas. Embora variações otimizadas incluam mecanismos para encerrar a execução mais cedo caso nenhuma troca seja feita durante uma

varredura, seu desempenho ainda permanece insatisfatório para grandes volumes de dados.

Por esses motivos, o Bubble Sort é raramente utilizado em aplicações reais que lidam com grandes conjuntos. Seu papel principal é didático: introduzir estudantes ao estudo da ordenação, ilustrar o conceito de algoritmos estáveis e demonstrar a diferença entre abordagens de ordenação simples e as mais avançadas. Em contextos limitados, como listas pequenas, situações de ensino ou em sistemas onde simplicidade de implementação é prioritária, o Bubble Sort ainda encontra alguma utilidade.

3.3 Selection Sort

O algoritmo *Selection Sort* apresenta uma abordagem diferente para resolver o problema da ordenação. Em vez de realizar múltiplas trocas por varredura, sua estratégia consiste em encontrar repetidamente o menor elemento da parte não ordenada e colocá-lo na posição correta. O processo segue até que todos os elementos estejam na ordem desejada.

Para ilustrar, imagine que você precise organizar cartas de baralho espalhadas sobre uma mesa. Em cada rodada, procura-se a carta de menor valor e a coloca na primeira posição. Depois, entre as cartas restantes, identifica-se a próxima menor e posiciona-se logo em seguida, e assim por diante, até que todas estejam alinhadas em ordem crescente. Essa mecânica reproduz fielmente o funcionamento do Selection Sort.

Uma característica importante do Selection Sort é que ele realiza exatamente $n - 1$ trocas, independentemente da configuração inicial da lista. Essa economia de movimentações pode ser vantajosa em cenários onde o custo de trocar elementos é alto. Contudo, o número de comparações é sempre o mesmo, independentemente da ordem da lista, o que limita sua adaptatividade. Isso significa que mesmo uma lista quase ordenada não trará benefícios de desempenho.

Assim como o Bubble Sort, o Selection Sort é um algoritmo *in-place*, não necessitando de estruturas auxiliares. Entretanto, ao contrário do Bubble Sort, ele não é estável em sua forma básica, pois a troca do menor elemento pode alterar a ordem relativa de itens iguais. Caso a estabilidade seja necessária, adaptações devem ser feitas, o que pode comprometer parte da simplicidade da implementação.

Na prática, o Selection Sort é considerado ineficiente para listas grandes, mas pode ser aplicado em casos específicos. Sua previsibilidade o torna útil em ambientes educacionais, para demonstrar algoritmos de ordenação, e em situações onde a minimização de trocas é mais relevante do que o número de comparações. Assim, embora não seja competitivo frente a algoritmos mais avançados, seu valor pedagógico e sua clareza justificam sua relevância histórica e acadêmica.

3.4 Shell Sort

O *Shell Sort*, introduzido por Donald Shell em 1959, é considerado uma das primeiras tentativas bem-sucedidas de melhorar algoritmos simples como o Insertion Sort. Sua proposta inovadora é utilizar intervalos (*gaps*) para comparar e ordenar elementos distantes no vetor, reduzindo gradualmente esses intervalos até chegar ao valor 1. No momento final, executa-se essencialmente um Insertion Sort, mas em uma lista já parcialmente organizada, o que torna o processo muito mais eficiente.

Podemos comparar o Shell Sort a organizar livros em uma grande estante. Em vez de arrumar um livro de cada vez na posição correta (como faz o Insertion Sort), primeiro divide-se a estante em blocos espaçados e organiza-se cada bloco. Depois, os blocos são refinados com intervalos menores até que todos os livros estejam perfeitamente alinhados. Esse método de ordenação progressiva permite uma redução significativa no número de movimentações.

Um ponto central do Shell Sort é a escolha da **sequência de gaps**, pois dela depende o desempenho prático do algoritmo. A sequência original, proposta por Shell, consiste em dividir o tamanho do vetor sucessivamente por dois, mas outras sequências mais sofisticadas foram propostas ao longo dos anos:

- **Sequência de Hibbard (1963):** definida pela fórmula $h_k = 2^k - 1$. Os primeiros valores são 1, 3, 7, 15, 31, Essa sequência distribui bem os intervalos e melhora o desempenho em relação à proposta original. Na prática, garante um pior caso de ordem $O(n^{3/2})$, mostrando-se mais eficiente que a versão básica.
- **Sequência de Pratt (1971):** formada por todos os números da forma $h = 2^i \cdot 3^j$, ordenados em ordem crescente. Os primeiros valores são 1, 2, 3, 4, 6, 8, 9, 12, Essa construção cobre uma grande variedade de intervalos, garantindo divisões equilibradas ao longo da execução. O pior caso dessa sequência chega a $O(n \log^2 n)$, bastante próximo de algoritmos mais eficientes.
- **Sequência de Sedgewick (1986):** uma das mais conhecidas e utilizadas, composta

por fórmulas híbridas envolvendo potências de 2 e 4, como $h_k = 4^i + 3 \cdot 2^{i-1} + 1$. Os primeiros valores são 1, 5, 19, 41, 109, Essa sequência foi desenhada para reduzir comparações redundantes, oferecendo um pior caso em torno de $O(n^{4/3})$. É considerada uma das melhores escolhas práticas para o Shell Sort.

Essas diferentes sequências demonstram a evolução do algoritmo e a busca por maior eficiência. Cada proposta melhora aspectos da ordenação, seja reduzindo o número de movimentações ou otimizando o equilíbrio entre os intervalos.

O Shell Sort é *in-place*, ou seja, não exige grandes quantidades de memória extra. Contudo, ele não é estável, pois a movimentação de elementos distantes pode alterar a ordem relativa de itens iguais. Apesar disso, em listas de tamanho moderado, costuma ser mais eficiente que Bubble Sort, Selection Sort e até mesmo Insertion Sort.

Na prática, o Shell Sort ocupa uma posição intermediária entre os algoritmos quadráticos mais simples e os algoritmos $O(n \log n)$. Ele é usado em contextos onde listas de tamanho médio precisam ser ordenadas de maneira rápida, sem a complexidade de algoritmos mais avançados. Por essa razão, tornou-se uma opção valiosa historicamente e ainda é encontrado em implementações modernas como uma solução prática e elegante.

3.5 Merge Sort

O *Merge Sort* é um dos algoritmos de ordenação mais importantes da história da ciência da computação, sendo considerado um marco no desenvolvimento das técnicas de divisão e conquista (*divide and conquer*). Ele foi criado em 1945 por *John von Neumann*, um dos precursores da computação moderna, e desde então se tornou uma das bases teóricas para a análise de desempenho de algoritmos de ordenação. Sua proposta inovadora consistia em dividir um problema grande em partes menores e mais simples de resolver, para depois combinar as soluções parciais em um resultado final eficiente e ordenado.

A ideia central do *Merge Sort* é simples e elegante: dividir recursivamente o vetor original em duas metades, até que cada sublista contenha apenas um elemento (que, por definição, já está ordenado). Em seguida, essas sublistas são mescladas (*merge*) de forma ordenada, reconstruindo o vetor completo em ordem crescente. Esse processo é repetido de forma recursiva até que todos os elementos estejam corretamente organizados. Essa abordagem garante que, mesmo para listas grandes e totalmente desordenadas, o número de comparações cresça de maneira logarítmica em relação ao tamanho da entrada, resultando em um dos algoritmos de ordenação mais eficientes já desenvolvidos.

Historicamente, o *Merge Sort* foi um dos primeiros algoritmos projetados com o objetivo de otimizar operações de entrada e saída em dispositivos de armazenamento (como fitas magnéticas), muito utilizados na época. Ele permitia dividir os dados em blocos menores, ordená-los separadamente e depois uni-los de forma eficiente, algo extremamente útil quando os computadores possuíam pouca memória principal. Essa característica de trabalhar bem com dados grandes, mesmo fora da memória principal, ainda hoje é aproveitada em sistemas de ordenamento externo (*external sorting*).

Do ponto de vista computacional, o *Merge Sort* é um algoritmo estável, ou seja, mantém a ordem relativa de elementos iguais, o que é uma vantagem em aplicações que dependem dessa propriedade, como sistemas de banco de dados. Além disso, seu comportamento é determinístico, apresentando desempenho praticamente idêntico independentemente da dis-

posição inicial dos dados — crescente, decrescente ou aleatória.

Entretanto, sua principal limitação está no uso de memória adicional, já que o processo de intercalação requer espaço auxiliar proporcional ao tamanho do vetor. Apesar disso, a previsibilidade e eficiência do **Merge Sort** o tornam uma das escolhas mais seguras e robustas para ordenações em larga escala, sendo amplamente utilizado até os dias atuais em implementações de bibliotecas e linguagens de programação modernas.

Em resumo, o *Merge Sort* combina simplicidade conceitual, rigor matemático e alta eficiência, representando um equilíbrio entre desempenho e clareza algorítmica. Seu impacto histórico e relevância prática o colocam entre os algoritmos mais estudados e aplicados da ciência da computação.

3.6 Quick Sort

O Quick Sort é um dos algoritmos de ordenação mais eficientes e amplamente utilizados na ciência da computação. Desenvolvido por Tony Hoare em 1960, ele se baseia na técnica de **divisão e conquista (divide and conquer)**, assim como o Merge Sort, mas apresenta uma abordagem diferente: em vez de combinar sublistas ordenadas, ele realiza a ordenação durante o próprio processo de divisão, o que o torna geralmente mais rápido e econômico em memória.

A ideia central do Quick Sort consiste em **escolher um elemento pivô** dentro do vetor e **particionar** os demais elementos em duas sublistas: uma contendo todos os valores menores ou iguais ao pivô e outra contendo os maiores. Após o particionamento, o pivô estará na sua posição final, e o processo é repetido recursivamente para as sublistas, até que toda a estrutura esteja ordenada.

Um ponto crucial no desempenho do Quick Sort é o método de **escolha do pivô**, pois ele influencia diretamente no balanceamento das partições. Se o pivô divide o conjunto de forma equilibrada, o algoritmo alcança o desempenho ótimo esperado. No entanto, divisões muito desbalanceadas podem gerar recursões profundas e aumentar o custo total da execução.

As principais estratégias de seleção de pivô são:

- **Primeiro elemento:** abordagem simples e tradicional, porém suscetível a piores casos quando o vetor já está ordenado ou quase ordenado.
- **Média de elementos (mediana de três):** usa a média de três valores — geralmente o primeiro, o central e o último — para tornar a divisão mais equilibrada.
- **Pivô aleatório:** escolhe o pivô de forma randômica, reduzindo a probabilidade de cair em piores casos sistemáticos.

O processo de particionamento é linear em relação ao tamanho do vetor, ou seja, executa em tempo $O(n)$, pois cada elemento é comparado ao pivô exatamente uma vez para decidir em

qual sublista será inserido. Após essa etapa, cada sublista é ordenada recursivamente, até restarem apenas elementos unitários.

Do ponto de vista de eficiência, o Quick Sort apresenta três cenários de análise:

Melhor caso: ocorre quando o pivô divide o vetor de forma equilibrada, criando duas sublistas de tamanhos aproximadamente iguais. A recorrência que representa o algoritmo é:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \quad (1)$$

Aplicando o Teorema Mestre, o custo total é:

$$T(n) = O(n \log n) \quad (2)$$

Caso médio: em entradas aleatórias, o particionamento tende a ser relativamente balanceado, e o custo esperado também é $O(n \log n)$. Este é o comportamento mais comum e justifica o excelente desempenho prático do algoritmo.

Pior caso: ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do vetor, resultando em uma divisão altamente desbalanceada. Nesse cenário, a recorrência se torna:

$$T(n) = T(n - 1) + O(n) \quad (3)$$

e o custo total cresce para:

$$T(n) = O(n^2) \quad (4)$$

Apesar dessa limitação teórica, o Quick Sort é considerado um dos algoritmos mais rápidos em aplicações reais, principalmente por ser um método *in-place* — utiliza apenas uma quantidade

constante de memória adicional — e por apresentar baixo custo de movimentação de dados. Além disso, sua implementação recursiva é direta e de fácil adaptação a diferentes contextos e tamanhos de entrada.

Diferentemente do Merge Sort, o Quick Sort não possui uma fase explícita de combinação, pois os elementos são organizados em suas posições corretas à medida que as partições são processadas. Isso o torna mais econômico em termos de espaço e mais adequado para ordenações internas (em memória principal).

Em resumo, o Quick Sort alia eficiência, elegância e praticidade, sendo amplamente utilizado em bibliotecas e sistemas modernos de ordenação. Seu desempenho médio $O(n \log n)$ e sua baixa exigência de memória fazem dele uma das soluções mais versáteis entre os algoritmos de ordenação, especialmente quando combinadas estratégias eficazes de escolha do pivô.

3.7 Heap Sort

O *Heap Sort* é um algoritmo de ordenação que se baseia em uma estrutura de dados específica chamada *heap*, geralmente implementada sobre um vetor. Um heap pode ser visto como uma árvore binária quase completa, organizada de forma a satisfazer uma propriedade de ordem: em um *max-heap*, cada nó possui valor maior ou igual ao de seus filhos; em um *min-heap*, cada nó possui valor menor ou igual ao de seus filhos. Essa organização garante que o maior (ou o menor) elemento da estrutura esteja sempre na raiz, isto é, na primeira posição do vetor.

Diferentemente de algoritmos como o *Insertion Sort* ou o *Bubble Sort*, que comparam e trocam elementos diretamente ao longo do vetor, o *Heap Sort* explora essa estrutura em forma de árvore para controlar de maneira mais eficiente qual elemento deve ser movido a seguir. No contexto de ordenação crescente, costuma-se utilizar um *max-heap*: o maior elemento do conjunto estará sempre na raiz, podendo ser facilmente removido e colocado na última posição livre do vetor. Em seguida, o heap é ajustado para que o próximo maior elemento volte a ocupar a raiz, e o processo se repete até que todos os elementos estejam ordenados.

A implementação típica do *Heap Sort* pode ser dividida em duas grandes etapas. Na primeira, o vetor original é transformado em um heap válido, processo conhecido como construção do heap. Nessa fase, os elementos são reorganizados de baixo para cima, ajustando a posição de cada nó para que a propriedade do heap seja satisfeita em toda a estrutura. Na segunda etapa, o algoritmo passa a extrair repetidamente o elemento da raiz (o maior, no caso de um *max-heap*) e a colocá-lo na parte final do vetor, que passa a representar a região já ordenada. A cada remoção da raiz, o heap tem seu tamanho lógico reduzido e precisa ser reorganizado por meio de um procedimento de manutenção que recoloca o novo elemento da raiz na posição correta dentro da árvore.

Esse procedimento de manutenção como descrito na figura 2 é comumente chamado de *heapify*. Intuitivamente, ele funciona empurrando o elemento que está na raiz (ou em um nó

interno) para baixo na árvore, trocando-o com o maior (ou menor, no caso de um *min-heap*) de seus filhos sempre que a propriedade do heap for violada. Esse processo desce nível a nível até que o nó esteja em uma posição coerente com a estrutura, restaurando a organização do heap. Embora local, esse ajuste garante que a ordem global seja preservada e que a raiz continue armazenando o elemento de maior prioridade. A ideia de heap está diretamente ligada

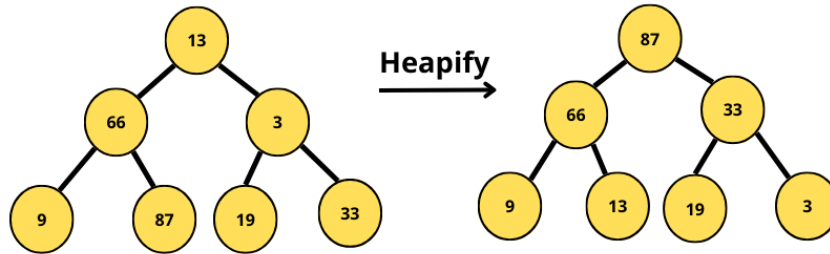


Figura 2: Descrição visual do algoritmo *Heapify*

ao conceito de fila de prioridade. Em uma fila de prioridade, cada elemento possui uma chave associada que representa sua prioridade, e as operações mais importantes são inserir novos elementos e remover o elemento de maior (ou menor) prioridade. Usando um *max-heap*, é possível implementar uma fila de prioridade na qual a operação de remoção corresponde à retirada do maior elemento situado na raiz. De modo análogo, um *min-heap* permite que o menor elemento seja sempre removido primeiro, sendo útil em algoritmos de escalonamento, simulação, eventos discretos e problemas de caminhos mínimos.

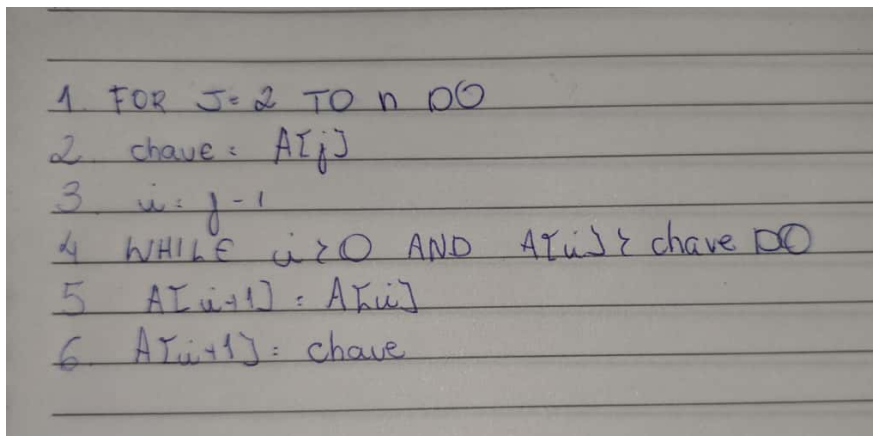
Na prática, o *Heap Sort* aproveita essas mesmas ideias da fila de prioridade: ele constrói um heap com todos os elementos do vetor e, em seguida, extrai o maior elemento de cada vez, reposicionando-o na porção final do vetor. Ao final desse processo, o vetor estará completamente ordenado. Além de apresentar um desempenho assintoticamente eficiente, o algoritmo tem a vantagem de ser *in-place*, isto é, não necessita de estruturas auxiliares complexas, já que todo o heap pode ser mantido dentro do próprio vetor original.

4 ANÁLISE DE COMPLEXIDADE

4.1 Insertion Sort

O algoritmo *Insertion Sort* percorre o vetor da esquerda para a direita, inserindo cada novo elemento na posição correta da porção já ordenada à sua esquerda. Isso é feito deslocando os elementos maiores para a direita e inserindo o elemento atual na posição adequada.

Pseudocódigo



```
1 FOR j=2 TO n DO
2 chave = A[j]
3 i = j-1
4 WHILE i > 0 AND A[i] > chave DO
5 A[i+1] = A[i]
6 A[i+1] = chave
```

Na primeira iteração, o segundo elemento do vetor é comparado com o primeiro e inserido na posição correta. Na segunda iteração, o terceiro elemento é comparado com os dois primeiros, sendo deslocados os elementos maiores e inserido o novo elemento na posição correta.

Esse processo se repete até que o último elemento seja inserido na posição correta, resultando em uma lista ordenada. Durante a execução:

- A sublista à esquerda do índice atual está sempre ordenada.
- O laço interno realiza comparações e deslocamentos até encontrar a posição correta.
- O número de deslocamentos depende da ordem inicial dos dados.

Tabela de Custos por Linha

linha	Custo	Número de vezes executada
1	C_1	n
2	C_2	$n-1$
3	C_3	$n-1$
4	C_4	$\sum_{j=2}^n t_j$
5	C_5	$\sum_{j=2}^n (t_j - 1)$
6	C_6	$\sum_{j=2}^n (t_j - 1)$
7	C_7	$\sum_{j=2}^n t_j$
8	C_8	$n-1$

Teste de mesa:

Insertion Sort: [5, 2, 9, 1, 6]						
iteração	j	chave = $A[j]$	$i = j-1$	$A[i] > \text{chave}$	$A[i+1] = A[i]$ $i=i-1$	lista
1	2	2	1	$5 > 2 \rightarrow T$	$A[2] = 5$	0 [2, 5, 9, 1, 6]
2	3	9	2	$5 < 9 \rightarrow F$	-	2 [2, 5, 9, 1, 6]
3	4	1	3	$9 > 1 \rightarrow T$	$A[4] = 9$	2 [1, 2, 5, 9, 6]
4	5	6	4	$9 > 6 \rightarrow T$	$A[5] = 9$	3 [1, 2, 5, 6, 9]
Final = [1, 2, 5, 6, 9]						

Expressão Geral de $T(n)$

$$T(n) = C_1 n + C_2 (n-1) + C_3 (n-1) + C_4 \sum_{j=2}^n t_j + C_5 \sum_{j=2}^n (t_j - 1) + C_6 \sum_{j=2}^n (t_j - 1) + C_7 \sum_{j=2}^n t_j + C_8 (n-1)$$

onde t_j representa o número de vezes que a condição do laço interno (linha 4) é avaliada na iteração j .

Expansão Algébrica dos Casos

Pior caso (lista em ordem decrescente):

$$T_j: j \rightarrow \sum_{j=2}^n T_j: \frac{n(n+1)}{2} - 1, \sum_{j=2}^n (T_j - 1) = \frac{n(n-1)}{2}$$

$$T(n) = \left(\frac{C_4 + C_5 + C_6 + C_7}{2} n^2 + (C_1 - C_2 - C_3 - C_8 + \frac{C_4 + C_7}{2} - \frac{C_5 + C_6}{2}) n + (-C_2 - C_3 - C_4 - C_7 - C_8) \right)$$

$$T_{\text{pior}}(n) = \Theta(n^2)$$

Melhor caso (lista já ordenada):

$$T_j: 1 \rightarrow \sum_{j=2}^n T_j: n-1, \sum_{j=2}^n (T_j - 1) = 0$$

$$T(n) = (C_1 + C_2 + C_3 + C_4 + C_7 + C_8) n + \text{constante}$$

$$T_{\text{melhor}}(n) = \Theta(n)$$

Caso médio (ordem aleatória):

$$E[T_j] = \frac{j+1}{2} \rightarrow \sum_{j=2}^n E[T_j]$$

$$T(n) = \left(\frac{C_4 + C_5 + C_6 + C_7}{2} n^2 + (C_1 - C_2 - C_3 - C_8 + \frac{C_4 + C_7}{2} - \frac{C_5 + C_6}{2}) n + (-C_2 - C_3 - C_4 - C_7 - C_8) \right)$$

$$T_{\text{médio}}(n) = \Theta(n)$$

- O *Insertion Sort* é um algoritmo **in-place**, ou seja, usa apenas $O(1)$ memória extra.
- É **estável**, preservando a ordem relativa de elementos iguais.

- Apesar de possuir complexidade quadrática na média e no pior caso, seu desempenho é excelente para listas pequenas ou quase ordenadas, pois o melhor caso é linear.

4.2 Bubble Sort

O *Bubble Sort* é um algoritmo de ordenação simples que compara pares de elementos adjacentes, trocando-os sempre que estão fora de ordem. A cada passagem completa pelo vetor, o maior elemento da parte não ordenada é deslocado para a sua posição correta.

Pseudocódigo:

```
1 FOR i = 0 TO n-1 DO
2   FOR j = 0 TO n-2 DO
3     IF A[j] > A[j+1]
4       SWAP A[j] AND A[j+1]
```

Tabela de custo por linhas:

Linhas	Descrição	Custo	Nº de execuções
1	Inicialização do loop	C1	n
2	Controle de loop interno	C2	$\sum_{i=1}^{n-1} (n-i)$
3	Comparação A[j] > A[j+1]	C3	$\sum_{i=1}^{n-1} (n-i)$
4	Troca de elementos	C4	$MC = 0$ $MC = \frac{n(n-1)}{4}$ $PC = \frac{n(n-1)}{2}$

Teste de mesa:

Bubble Sort [5, 2, 9, 1, 6]

Itens	Itens	$A[j] > A[j+1]$	$A[j] \leftrightarrow A[j+1]$	Iter
1	1	$5 > 2 \rightarrow T$	$5 \leftrightarrow 2$	[2, 5, 9, 1, 6]
1	2	$5 > 9 \rightarrow F$	-	"
1	3	$9 > 1 \rightarrow T$	$9 \leftrightarrow 1$	[2, 5, 1, 9, 6]
1	4	$9 > 6 \rightarrow T$	$9 \leftrightarrow 6$	[2, 5, 1, 6, 9]
2	1	$2 > 5 \rightarrow F$	-	"
2	2	$5 > 1 \rightarrow T$	$5 \leftrightarrow 1$	[2, 1, 5, 6, 9]
2	3	$5 > 6 \rightarrow F$	-	"
3	1	$2 > 1 \rightarrow T$	$2 \leftrightarrow 1$	[1, 2, 5, 6, 9]
3	2	$2 > 5 \rightarrow F$	-	"
4	1	$1 > 2 \rightarrow F$	-	"

Final: [1, 2, 5, 6, 9]

Melhor caso (lista já ordenada):

- O algoritmo percorre o vetor e verifica que nenhuma troca é necessária.
- Ainda assim, ele precisa realizar as comparações para verificar se os elementos estão na ordem correta.

Número de comparações: $C(n) = (n-1)$

Número de trocas: $T(n) = 0$

Tempo total

$$T(n) = C_1 n + C_2 (n-1)$$

Simplificação:

$$T(n) = (C_1 + C_2) n - C_2$$

O que resulta em uma complexidade $\mathcal{O}(n)$ para o melhor caso.

Pior caso (lista inversa):

O algoritmo precisa realizar o número máximo de trocas em cada iteração, elevando seu custo operacional de maneira expressiva.

Número de comparações:

$$C(n) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

Número de trocas:

$$Trocas(n) = \frac{n(n-1)}{2}$$

Tempo total:

$$T(n) = C_1 n + C_2 \frac{n(n-1)}{2}$$

Simplificação:

$$T(n) = \frac{C_2}{2} n^2 + \left(C_1 - \frac{C_2}{2} \right) n$$

Resultando em uma complexidade quadrática $O(n^2)$ para o pior caso.

Caso médio:

No caso médio (lista aleatória), é esperado que cerca de metade das comparações resultem em trocas.

Número de comparações (igual ao pior caso):

$$C(n) = \frac{n(n-1)}{2}$$

Número médio de trocas:

$$Trocas(n) \approx \frac{n(n-1)}{4}$$

Tempo total:

$$T(n) = C_1 n + C_2 \frac{n(n-1)}{4}$$

Simplificação:

$$T(n) = \frac{C_2}{4} n^2 + \left(C_1 - \frac{C_2}{4} \right) n$$

Resultando em uma complexidade quadrática $\mathcal{O}(n^2)$ também para o caso médio.

O *Bubble Sort*, apesar de sua clareza conceitual e valor didático, apresentou desempenho inferior em todas as categorias de teste. Mesmo otimizações não reduzem significativamente sua complexidade quadrática, o que o torna inadequado para aplicações práticas, sendo mais indicado para fins de ensino e ilustração dos princípios básicos de ordenação.

4.3 Selection Sort

O algoritmo *Selection Sort* funciona escolhendo repetidamente o menor elemento da parte não ordenada do vetor e trocando-o de posição com o elemento que ocupa a posição atual da iteração. Dessa forma, a cada passo um novo elemento é colocado na sua posição definitiva, até que toda a lista esteja ordenada.

Pseudocódigo:

```

1. FOR  $i = 0$  TO  $n-1$  DO
2.    $min = i$ 
3.   FOR  $j = i+1$  TO  $n-1$  DO
4.     IF  $A[j] < A[min]$ 
5.        $min = j$ 
6.   SWAP  $A[i]$  AND  $A[min]$ 
  
```

Tabela de custo por linha:

Linha	Descrição	Custo	nº Execuções
1	Controle do laço externo	C1	n
2	Atribuição $min = i$	C2	n
3	Controle do laço interno	C3	$\sum_{i=0}^{n-1} (n-i)$
4	Comparação $A[j] < A[min]$	C4	$\sum_{i=0}^{n-1} (n-i-1)$
5	Atribuição do índice min	C5	$\frac{n(n-1)}{2}$ (pior caso) mínimo: 0 (melhor caso)
			mitaço metade
6	Troca de elementos	C6	

Teste de mesa:

Selection Sort: [5, 2, 9, 1, 6]

i	j	A[j]	A[j] < A[min]	min	A[j] ↔ A[min]	data
1	2	2	2 < 5 → T	2	-	[5, 2, 9, 1, 6]
1	3	9	9 < 2 → F	-	-	"
1	4	1	1 < 2 → T	4	-	"
1	5	6	6 < 1 → F	-	-	"
1	-	-	-	-	5 ↔ 1	[1, 2, 9, 5, 6]
2	3	9	9 < 2 → F	-	-	"
2	4	5	5 < 2 → F	-	-	"
2	5	6	6 < 2 → F	-	-	"
2	-	-	-	-	2 ↔ 2	[1, 2, 9, 5, 6]
3	4	5	5 < 9 → T	4	-	"
3	5	6	6 < 5 → F	-	-	"
3	-	-	-	-	9 ↔ 5	[1, 2, 5, 9, 6]
4	5	6	6 < 9 → T	5	-	"
4	-	-	-	-	9 ↔ 6	[1, 2, 5, 6, 9]
Final: [1, 2, 5, 6, 9]						

Melhor caso:

No melhor caso, o vetor já está ordenado. Ainda assim, o algoritmo precisa comparar cada elemento da parte não ordenada para verificar se existe algum menor. Como essa verificação ocorre a cada iteração, o número de comparações continua o mesmo, mas as trocas deixam de ser necessárias, pois o índice do menor elemento nunca muda. Isso significa que, embora haja menos movimentações, o esforço total do algoritmo não melhora significativamente, mantendo-se quadrático.

① número de comparações é dado por:

$$C(n) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

Logo:

$$T(n) = C_1 n + C_2 \frac{n(n-1)}{2} \rightarrow T(n) = \frac{C_2 n^2}{2} + \left(C_1 - \frac{C_2}{2}\right)n$$

Portanto, mesmo no cenário ideal, o algoritmo ainda possui complexidade $\mathcal{O}(n^2)$.

Pior caso:

No pior caso, o menor elemento de cada iteração está sempre na última posição da parte não ordenada. Isso obriga o algoritmo a atualizar o índice do mínimo a cada comparação, além de realizar uma troca em todas as passagens. Dessa forma, o número de comparações permanece o mesmo, mas o número de trocas é máximo. Ainda assim, o fator predominante é o custo das comparações, que garante que o tempo de execução seja da ordem quadrática. Logo, o pior caso também é $\mathcal{O}(n^2)$.

Caso médio:

No caso médio (lista aleatória), o algoritmo também percorre todas as posições em busca do menor elemento. O que muda é que em algumas iterações haverá troca e em outras não. Entretanto, como as trocas são poucas em relação ao número de comparações, o tempo total se mantém próximo ao do pior caso. Assim, tem-se que $T(n) = \Theta(n^2)$.

O *Selection Sort* manteve desempenho constante independentemente da disposição inicial dos dados, realizando sempre o mesmo número de comparações. Embora minimize o número de trocas, seu custo quadrático o torna pouco eficiente para grandes listas. Ainda assim, é útil em contextos educacionais e em situações onde a previsibilidade e a simplicidade são mais relevantes que a velocidade.

4.4 Shell Sort

O *Shell Sort* utiliza uma sequência de intervalos (*gaps*) para ordenar elementos distantes entre si, diminuindo gradualmente os *gaps* até chegar a 1. Isso reduz a quantidade de deslocamentos necessários no *Insertion Sort* tradicional.

Pseudocódigo:

```

1. gap = n/2
2. WHILE gap > 0
3.   FOR i = gap TO n-1 DO
4.     aux = A[i]
5.     j = i
6.     WHILE j > gap AND A[j-gap] > aux
7.       A[j] = A[j-gap]
8.       j = j - gap
9.     A[j] = aux
10.  gap = gap / 2
  
```

Tabela de custo por linhas:

Linhas	Descrição	Custo	Nº. comparações
1	Inicialização do gap	C1	$\log^2 n$
2	Teste do loop externo	C2	$\log^2 n$
3	Loop para inserir elementos	C3	$n \log^2 n$
4-5	Inicializações	C4	$n \log^2 n$
6	Comparações internas	C5	depende do gap, total $\leq n^2$
7-8	Movimentações	C6	$n \log^2 n$
9	Atribuição final	C7	$n \log^2 n$
10	Redução do gap	C8	$\log^2 n$

Teste de mesa:

Shell Sort [5, 2, 9, 1, 6]

Gap	i	temp = A[i]	j	A[j-gap] > temp	A[i] = A[j-gap]	A[j] = temp
2	3	0	3	5 > 0 → F	-	-
2	4	1	4	2 > 1 → T	A[4] = 2	2
2	-	-	0	j < gap	-	-
2	5	6	5	2 > 6 → F	-	-
1	2	5	2	1 > 5 → F	-	-
1	3	9	3	5 > 9 → F	-	-
1	4	2	4	9 > 2 → T	A[4] = 9	3
1	-	-	2	1 > 2 → F	-	-
1	5	6	5	9 > 6 → T	A[5] = 9	4
1	-	-	4	6 = A[4]	-	-

Final = [1, 2, 5, 6, 9]

Melhor caso:

- Nesse cenário, as comparações ainda acontecem, mas poucas movimentações são necessárias.
- Dependendo da sequência de *gaps* escolhida, o melhor caso pode ser expresso como:

$$T(n) = O(n \log n) \quad (c_2 + c_3 + c_4 + c_7) n \log n$$

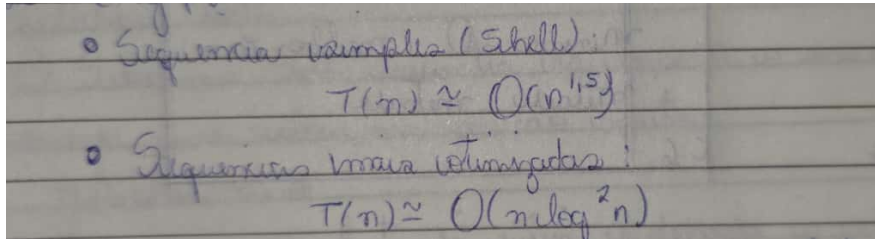
Pior caso:

A análise do pior caso depende da sequência de *gaps*:

- Sequência de Shell ($n/2, n/4, \dots, 1$):
 $T(n) = O(n^2)$
- Sequência de Hibbard (2^{k-1}):
 $T(n) = O(n^{3/2})$
- Sequência de Sedgewick (empírica, baseada em combinações de potências de 3):
 $T(n) = O(n^{3/2})$

Caso médio:

No caso médio, considera-se uma entrada aleatória, com diferentes sequências de *gaps*:



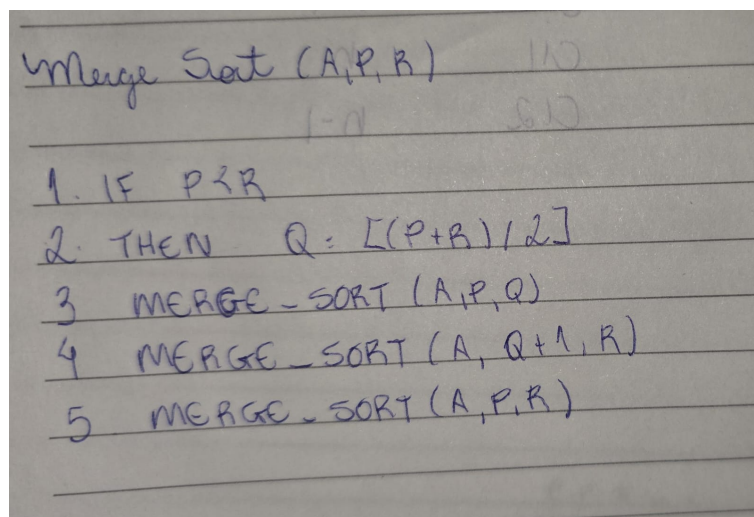
Assim, o *Shell Sort* apresenta desempenho bastante variável, mas geralmente superior ao *Insertion Sort*, *Bubble Sort* e *Selection Sort* para listas grandes.

O *Shell Sort* apresentou resultados superiores em relação aos demais algoritmos quadráticos, destacando-se pela redução significativa no tempo de execução, principalmente em listas grandes e aleatórias. A eficiência do algoritmo está diretamente ligada à escolha da sequência de *gaps*, o que lhe confere um bom equilíbrio entre desempenho e simplicidade, tornando-o uma alternativa prática em cenários intermediários.

4.5 Merge Sort

O *Merge Sort* divide o vetor em duas metades, ordena recursivamente cada metade e intercala (*merge*) as duas listas ordenadas em tempo linear no tamanho do subproblema. A etapa de intercalação sempre percorre todos os elementos a serem mesclados, independentemente da ordem inicial (crescente, decrescente ou aleatória).

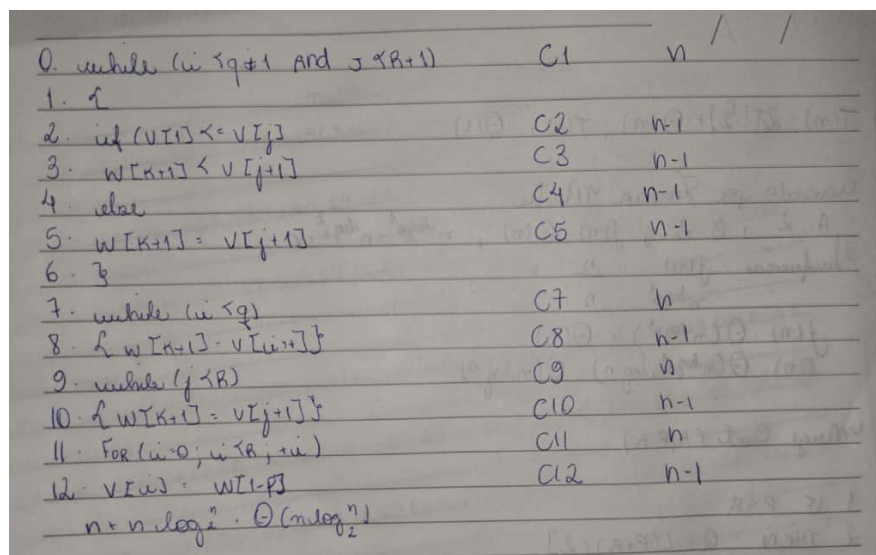
Pseudocódigo:



```

Merge Sort (A, P, R)
1. IF P < R
2. THEN Q = [(P+R)/2]
3. MERGE-SORT (A, P, Q)
4. MERGE-SORT (A, Q+1, R)
5. MERGE-SORT (A, P, R)
  
```

Tabela de custo por linha:



0. while (i < q+1 and j < R+1)	C1	n
1. i		
2. if (V[i] < V[j])	C2	n-1
3. W[k+1] = V[j+1]	C3	n-1
4. else	C4	n-1
5. W[k+1] = V[i+1]	C5	n-1
6. i		
7. while (i < q)	C7	n
8. { W[k+1] = V[i+1] }	C8	n-1
9. while (j < R)	C9	n
10. { W[k+1] = V[j+1] }	C10	n-1
11. For (i=0; i < R; i++)	C11	n
12. V[i] = W[i]	C12	n-1
$n + n \log_2 \cdot \Theta(n \log_2^2)$		

Teste de mesa:

Merge (A, P, q, r)

Passo	1	2	3	4	5	6	7	8
1	1321132 → T	E[1]: 2 < D[1]: 3 → T	A[1]: E[1]: 2	K: K+1	2	1	2	
2	2321132 → T	E[2]: 8 < D[2]: 3 → F	A[2]: D[2]: 3	K: K+1	2	2	3	
3	2321232 → T	E[2]: 8 < D[2]: 5 → F	A[3]: D[2]: 5	K: K+1	2	3	4	
4	2321332 → F	-	-	-	3	3	4	
5	2321332 → F	-	-	-	3	3	4	
6	2321332 → F	-	-	-	3	3	4	
7	2321332 → F	-	-	-	3	3	4	
8	2321332 → F	-	-	-	3	3	4	

Vetor final [2, 3, 5, 8]

Recorrência canônica:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1)$$

Provando por Teorema Mestre:

A: 2 ; B: 2 ; $f(n) = \Theta(n)$; $n^{\log_B A} = n^{\log_2 2} = n$

Comparação: $\frac{f(n)}{n^{\log_B A}} = \frac{n}{n} = 1$

$f(n) = \Theta(n^{\log_B A}) = \Theta(n)$

$T(n) = \Theta(n^{\log_B A} \log n) = \Theta(n \log n)$

Melhor Caso:

- **Intuição:** Mesmo se as metades já estiverem “favoráveis”, a rotina de *merge* precisa olhar para **todos os elementos** de ambas as metades e escrever cada um exatamente uma vez. Assim, o custo por nível permanece linear.
- **Recorrência:** Idêntica ao caso geral, pois a etapa de merge continua $\Theta(n)$.
- **Resultado:**

$$T_{\text{melhor}}(n) = \Theta(n \log n)$$

Pior Caso:

- **Intuição:** Quando os elementos de uma metade “competem” com a outra em quase todas as comparações, ainda assim o merge visita todos os itens e faz um número de comparações linear. Cada nível da árvore de recursão custa $\Theta(n)$, e há $\Theta(\log n)$ níveis.
- **Recorrência:** Mesma forma $2T(n/2) + \Theta(n)$.
- **Resultado:**

$$T_{\text{pior}}(n) = \Theta(n \log n)$$

Médio Caso:

- **Intuição:** Para entradas aleatórias, a quantidade esperada de comparações feita pela intercalação permanece linear por nível, e a profundidade da recursão é $\Theta(\log n)$. A ordem de crescimento não muda com a distribuição.
- **Recorrência:** Mesma forma $2T(n/2) + \Theta(n)$.
- **Resultado:**

$$T_{\text{médio}}(n) = \Theta(n \log n)$$

4.6 Quick Sort

Pseudocódigo:

```
QUICKSORT(A, P, R)
  IF  $p \leq R$ 
     $Q \leftarrow \text{PARTICIONA}(A, P, R)$ 
    QUICKSORT(A, P,  $Q-1$ )
    QUICKSORT(A,  $Q+1$ , R)

PARTICIONA(A, P, R)
  PIVOT  $\leftarrow A[R]$ 
   $i \leftarrow p-1$ 
  FOR  $j \leftarrow p$  até  $R-1$  DO
    IF  $A[j] \leq \text{PIVOT}$ 
       $i \leftarrow i+1$ 
      SWAP  $A[i] \leftrightarrow A[j]$ 
  SWAP  $A[i+1] \leftrightarrow A[R]$ 
  RETURN  $i+1$ 
```

Tabela de custo por linha:

Linha	Ação	Custo	Nº execuções
1	Verificação condição $p \leq R$	C_1	n
2	PARTICIONA	C_2	n
3-4	Chamadas recursivas	C_3	diversas (depende da partição)
5-12	Loop interno	C_4	$n-1$
13	Troca final do pivô	C_5	1

Teste de mesa:

Passo	Elemento (P, R)	Subvetor	Pivot	i	j	Particiona	Ação	Alterar
1	(1, 5)	9, 3, 7, 1, 6	6	1	0	0 < 6? Não	-	-
2	(1, 5)	9, 3, 7, 1, 6	6	2	0	3 < 6? Sim	$A[1] \leftrightarrow A[2]$	-
3	(1, 5)	9, 3, 7, 1, 6	6	3	1	7 < 6? Não	-	-
4	(1, 5)	9, 3, 7, 1, 6	6	4	1	1 < 6? Não	$A[2] \leftrightarrow A[4]$	-
5	(1, 5)	-	6	-	-	-	$A[3] \leftrightarrow A[5]$	-
6	-	-	-	-	-	-	-	-
7	(1, 2)	3, 1	1	1	0	0 < 0	-	-
8	(1, 2)	-	1	-	-	-	$A[1] \leftrightarrow A[2]$	-
9	-	-	-	-	-	-	-	-
10	(4, 5)	9, 7	7	4	3	3 < 3	-	-
11	(4, 5)	-	7	-	-	-	$A[4] \leftrightarrow A[5]$	-
12	-	-	-	-	-	-	-	-
13	-	-	-	-	-	-	-	-

Lista ordenada:

1	9, 3, 7, 1, 6	5, 3, 1, 6, 9, 7
2	9, 3, 7, 1, 6	7, 3, 1, 6, 9, 7
3	9, 3, 7, 1, 6	8, 1, 3, 6, 9, 7
4	9, 3, 1, 6, 9, 7	

Expressão geral da recorrência:

$$T(n) = T(k) + T(n-k-1) + \Theta(n)$$

onde:

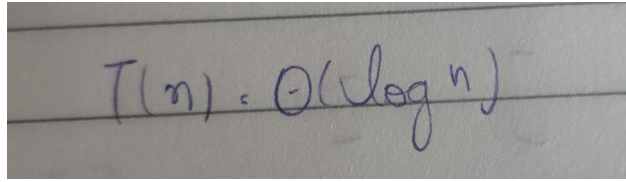
- $T(k)$ representa o tempo para ordenar a sublista à esquerda do pivô;
- $T(n-k-1)$ representa o tempo para ordenar a sublista à direita;
- $\Theta(n)$ é o custo linear da função PARTICIONA.

Melhor Caso:

Ocorre quando o pivô divide o vetor de forma equilibrada, gerando duas sublistas de tamanhos próximos. Nesse cenário:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

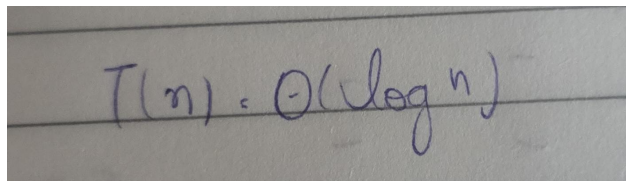
Aplicando o Teorema Mestre, temos:


$$T(n) = O(\log n)$$

Portanto, no melhor caso, o algoritmo tem crescimento log-linear e apresenta desempenho ótimo.

Caso Médio:

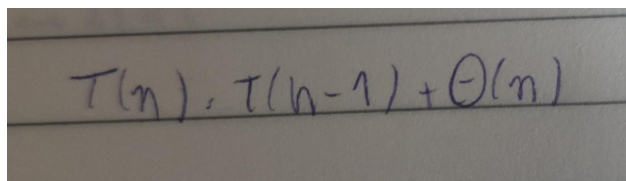
No caso médio, assume-se que o pivô divide o vetor de maneira razoavelmente balanceada, o que, estatisticamente, ocorre na maioria das vezes. Assim, a recorrência é similar à do melhor caso e resulta em:


$$T(n) = O(\log n)$$

Esse é o comportamento predominante em aplicações práticas, justificando o uso do Quick Sort em sistemas reais e bibliotecas padrão de linguagens de programação.

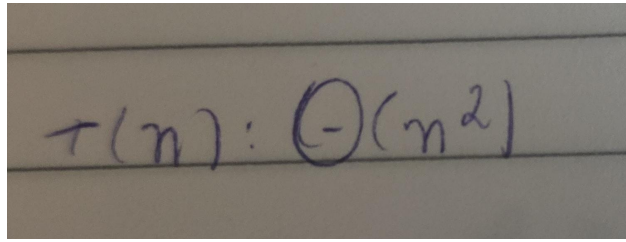
Pior Caso:

O pior caso ocorre quando o pivô é sempre o menor ou o maior elemento, gerando partições extremamente desbalanceadas. A recorrência torna-se:


$$T(n) = T(n-1) + O(n)$$

que se resolve para:

Esse cenário pode acontecer, por exemplo, quando a entrada já está ordenada e o pivô escolhido é o primeiro ou o último elemento.

A photograph of a piece of lined paper with the handwritten equation $T(n) = O(n^2)$ in blue ink.

O procedimento de particionamento possui complexidade linear $O(n)$, pois percorre o vetor uma única vez, comparando cada elemento ao pivô.

O Quick Sort é *in-place*, exigindo apenas $O(1)$ espaço adicional.

Não há fase explícita de combinação, uma vez que a ordenação ocorre durante o processo de divisão.

O algoritmo não é estável, pois pode inverter a ordem de elementos iguais durante as trocas.

O Quick Sort combina eficiência e simplicidade, oferecendo o melhor equilíbrio entre desempenho e uso de memória em ordenações internas. Embora seu pior caso seja $O(n^2)$, implementações modernas empregam estratégias de escolha de pivô — como a mediana de três ou pivôs aleatórios — que reduzem significativamente a probabilidade de divisões desbalanceadas, tornando o algoritmo uma das soluções mais eficazes para grandes volumes de dados.

4.7 Heap Sort

O *Heap Sort* realiza a ordenação reorganizando o vetor de forma progressiva, de modo que o elemento de maior prioridade seja isolado e reposicionado em sua posição definitiva a cada iteração. O processo inicia com a formação de um heap a partir da disposição atual dos dados, garantindo que o elemento prioritário esteja posicionado na raiz da estrutura. Uma vez que esse elemento é identificado, ele é trocado com a última posição ainda não ordenada do vetor, e o tamanho útil da estrutura é reduzido. Em seguida, o heap é ajustado localmente para restaurar sua organização e revelar o próximo elemento a ser movido. Esse ciclo se repete até que toda a região não ordenada seja eliminada, o que produz, ao final, um vetor completamente ordenado. O algoritmo, portanto, trabalha desmontando o heap ao mesmo tempo em que estabelece a ordem crescente ou decrescente dos dados, dependendo da variação utilizada.

Pseudocódigo:

```
min-heapify (A, i, heapsize)
    left ← 2 * i
    right ← 2 * i + 1
    maior ← i

    se left <= heapsize e A[left] > A[maior] então
        maior ← left
    se right <= heapsize e A[right] > A[maior] então
        maior ← right
    fim-se

    se maior ≠ i então
        trocar A[i] ↔ A[maior]
        min-heapify (A, maior, heapsize)
    fim-se
```

```

BUILD-MIN-HEAP(A, n)
  heapsize ← n
  para i ← [n/2] down to 1
    MIN-HEAPIFY(A, i, heapsize)

```

```

HEAP-SORT min(A, n)
  BUILD-MIN-HEAP(A, n)
  para i ← n down to 2
    troca A[1] ↔ A[i]
    heapsize ← heapsize - 1
    MIN-HEAPIFY(A, 1, heapsize)

```

Filas de prioridade

A fila de prioridade é uma estrutura projetada para manipular elementos de acordo com um critério de importância, permitindo que a remoção sempre selecione o elemento mais prioritário. No caso do min-heap, essa prioridade corresponde ao menor valor armazenado. Essa característica é mantida graças à organização estrutural do heap, que garante que a menor chave sempre permaneça na raiz. Assim, operações como extração do mínimo, diminuição de chaves e inserção de novos valores podem ser realizadas de forma eficiente, com acessos previsíveis e baixo custo.

A remoção do elemento prioritário ocorre substituindo a raiz pelo último elemento da região ativa e aplicando um ajuste local, de forma que a propriedade do heap seja restabelecida. Na inserção, cria-se inicialmente uma posição sentinela com valor infinito, que é substituído pela chave real e reposicionado até encontrar a posição compatível com sua prioridade. A operação de diminuição de chave segue a mesma lógica: ao atualizar o valor, o elemento

sobe pela estrutura enquanto sua prioridade for superior à dos nós acima. Essa integração entre organização estrutural e manutenção eficiente da hierarquia faz da fila de prioridade uma ferramenta essencial em algoritmos clássicos como Dijkstra, Prim e diversas rotinas de escalonamento.

HEAP-MINIMUM (A)
RETORNAR A[1]

HEAP-EXTRACT-MIN (A, heapsize)
use heapsize ≥ 1 então
como "heap do zero"
 $\text{min} \leftarrow A[1]$
 $A[1] \leftarrow A[\text{heapsize}]$
 $\text{heapsize} \leftarrow \text{heapsize} - 1$
 $\text{MIN-HEAPIFY}(A, 1, \text{heapsize})$
retornar min

HEAP-DECREASE-KEY (A, i, novachave)
use novachave $\leq A[i]$ então
como "nova chave maior que a atual"
 $A[i] \leftarrow \text{novachave}$
enquanto $i > 1$ e $A[\lfloor i/2 \rfloor] > A[i]$ faça
trocar $A[i] \leftrightarrow A[\lfloor i/2 \rfloor]$
 $i \leftarrow \lfloor i/2 \rfloor$

```

min-HEAP-INSERT (A, heapsize, chave)
    heapsize  $\leftarrow$  heapsize + 1
    A[heapsize]  $\leftarrow$  + $\infty$ 
    HEAP-DECREASE-KEY (A, heapsize, chave)

```

Tabela de custo por linha:

Linhas	Comando	Custo	nº de comparações
1	BUILD-MIN-HEAP (A, N)	C1	1
2	for $i \leftarrow n$ down to 2	C2	$(n-1)$
3	trocar A[i] $\leftrightarrow$ A[1]	C3	$n-1$
4	heapsize $\leftarrow$ heapsize - 1	C4	$n-1$
5	min-HEAPIFY (A, 1, heapsize)	C5	$n-1$

Fórmula Geral da Recorrência

A etapa dominante do Heap Sort é a fase de ordenação, na qual cada extração do elemento prioritário exige a aplicação do procedimento de manutenção na raiz. Esse comportamento pode ser descrito pela seguinte recorrência:

$$T(n) = T(n - 1) + O(\log n)$$

Somando-se todas as alturas percorridas durante as chamadas ao procedimento *heapify*,

obtem-se:

$$T(n) = \sum_{k=1}^n \log k = \Theta(n \log n)$$

Portanto, a complexidade assintótica do algoritmo é dominada pela etapa de extrações sucessivas, que acumulam custo proporcional a $n \log n$.

Análise do Melhor, Médio e Pior Caso

Melhor Caso. O Heap Sort não apresenta variação significativa no melhor caso. Mesmo que o vetor de entrada já esteja ordenado ou parcialmente organizado, todas as iterações da fase de ordenação necessitam de uma chamada ao procedimento *heapify* na raiz, e a construção inicial do heap continua analisando todos os nós internos. Dessa forma, a altura percorrida em cada chamada permanece proporcional a $\log n$, resultando em:

$$T_{\text{melhor}}(n) = \Theta(n \log n)$$

Caso Médio. No caso médio, assume-se que a posição ocupada pelo elemento deslocado para a raiz após cada troca é aleatória. Assim, o procedimento de manutenção percorre, em média, uma fração constante da altura total da estrutura. Como essa operação é realizada aproximadamente n vezes, a soma dos custos resulta novamente em uma complexidade proporcional a $n \log n$:

$$T_{\text{médio}}(n) = \Theta(n \log n)$$

Pior Caso. Ocorre quando, após cada extração, o elemento reposicionado na raiz precisa descer até o último nível do heap. Esse cenário maximiza o custo individual das chamadas ao procedimento de manutenção. No entanto, como a altura da árvore permanece limitada

por $\log n$ e o processo é repetido para todas as extrações, o custo total mantém-se dentro da mesma ordem de grandeza dos demais casos:

$$T_{\text{pior}}(n) = \Theta(n \log n)$$

O Heap Sort mantém a mesma complexidade nos três cenários — melhor, médio e pior caso — pois a estrutura do heap exige que as operações estruturais percorram uma altura logarítmica independentemente da ordem inicial dos dados. Por isso, trata-se de um algoritmo estável em desempenho, previsível e eficiente para grandes volumes de dados, além de operar com uso mínimo de memória adicional.

5 TABELA E GRÁFICO

5.1 Insertion Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000001	0,000002	0,000013	0,000144	0,001397	0,014323
Decrescente	0,000001	0,000025	0,001855	0,203572	17,053548	2114,295511
Aleatório	0,000001	0,000016	0,000916	0,085906	15,17523	1243,076734

Figura 3: Tabela de tempo por segundo do algoritmo *Insertion Sort*

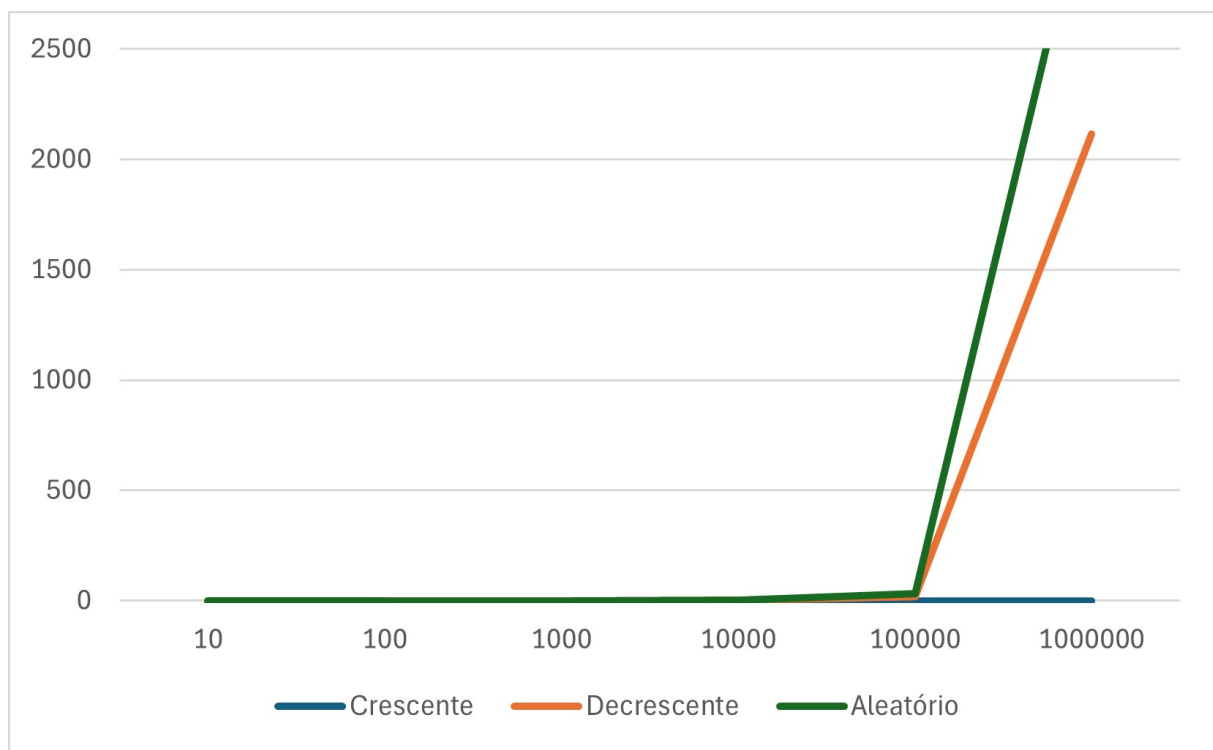


Figura 4: Gráfico de tempo por segundo do algoritmo *Insertion Sort*

Quando aplicado a conjuntos de dados pequenos, o algoritmo apresenta um desempenho bastante eficiente. Contudo, em entradas maiores, devido ao seu custo quadrático $\Theta(n^2)$, o tempo de processamento cresce de forma acentuada.

5.2 Bubble Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000319	0,000342	0,002893	0,257234	22,934024	3282,021746
Decrescente	0,000407	0,000376	0,001855	0,241055	22,622774	3718,897258
Aleatório	0,000439	0,000701	0,003165	0,242464	32,565813	3409,042193

Figura 5: Tabela de tempo por segundo do algoritmo *Bubble Sort*

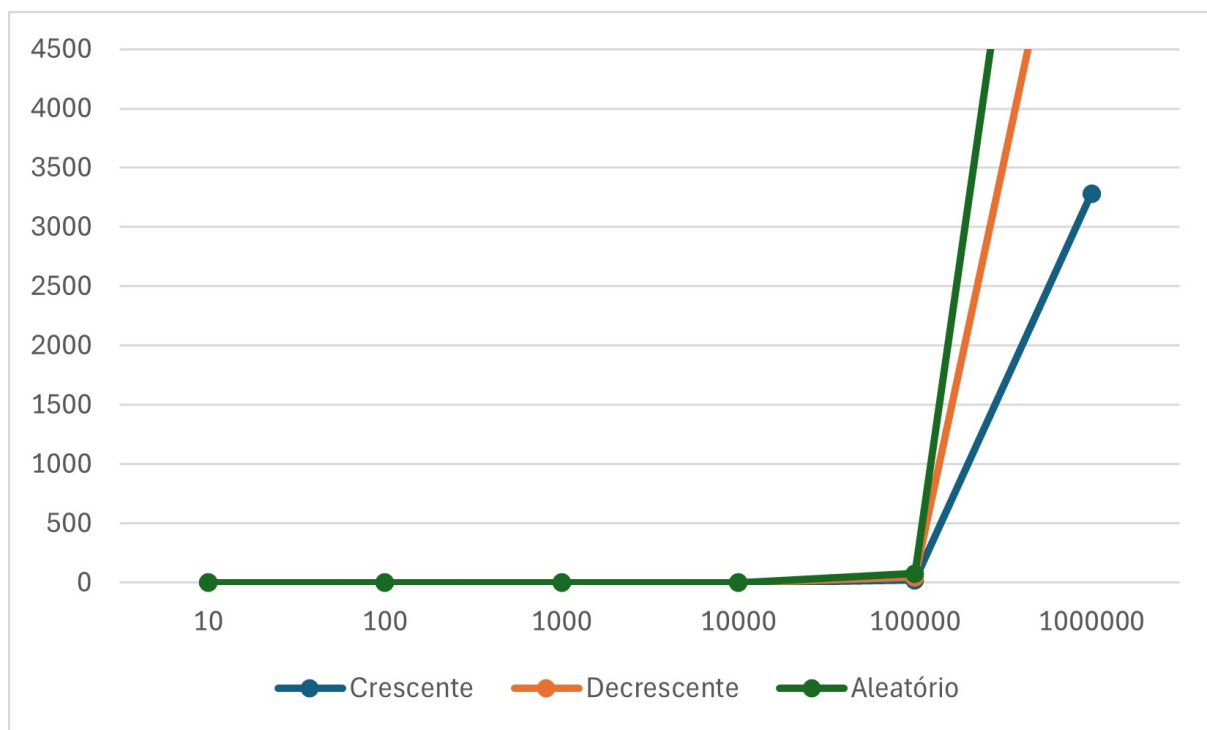


Figura 6: Gráfico de tempo por segundo do algoritmo *Bubble Sort*

5.3 Selection Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000266	0,000348	0,002002	0,131247	12,901212	1338,550982
Decrescente	0,00023	0,000324	0,001915	0,143501	13,907248	1438,595122
Aleatório	0,000244	0,000328	0,001860	0,132509	12,668543	1339,310597

Figura 7: Tabela de tempo por segundo do algoritmo *Selection Sort*

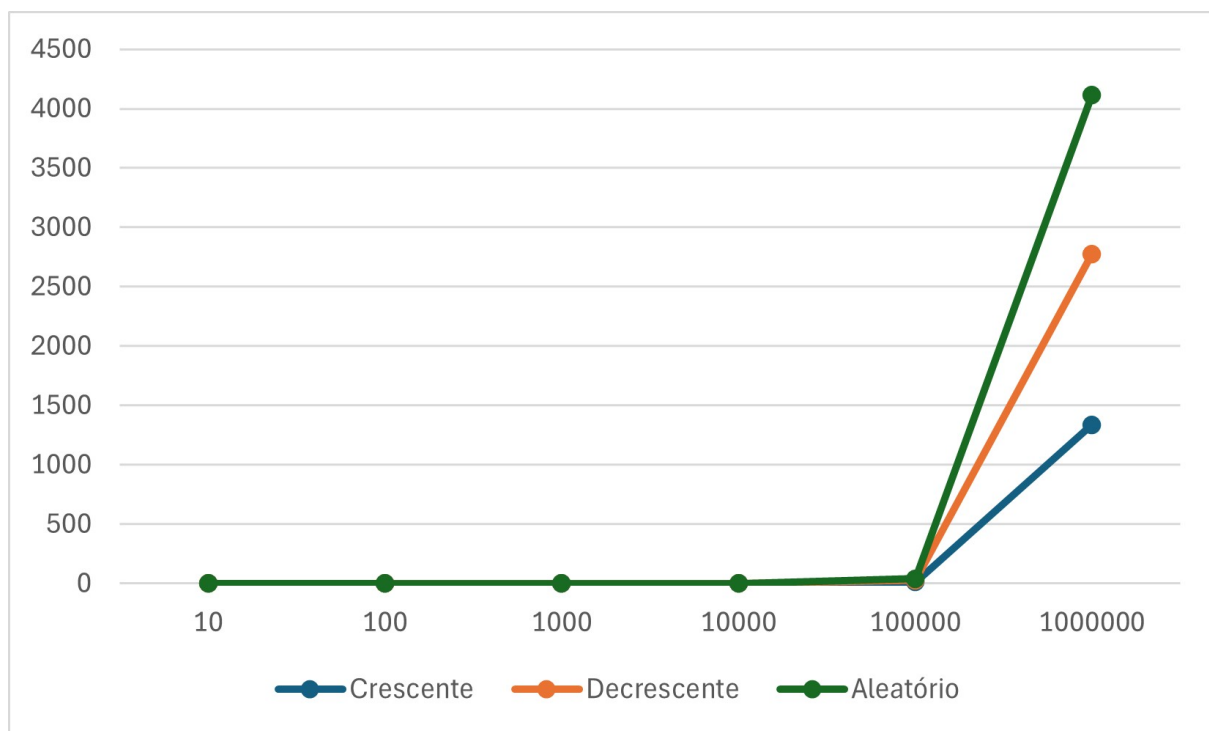


Figura 8: Gráfico de tempo por segundo do algoritmo *Selection Sort*

5.4 Shell Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000261	0,000301	0,000367	0,000837	0,008615	0,073479
Decrescente	0,000278	0,000300	0,000407	0,001122	0,010887	0,120733
Aleatório	0,000260	0,000284	0,000422	0,002766	0,033935	0,421096

Figura 9: Tabela de tempo por segundo do algoritmo *Shell Sort*

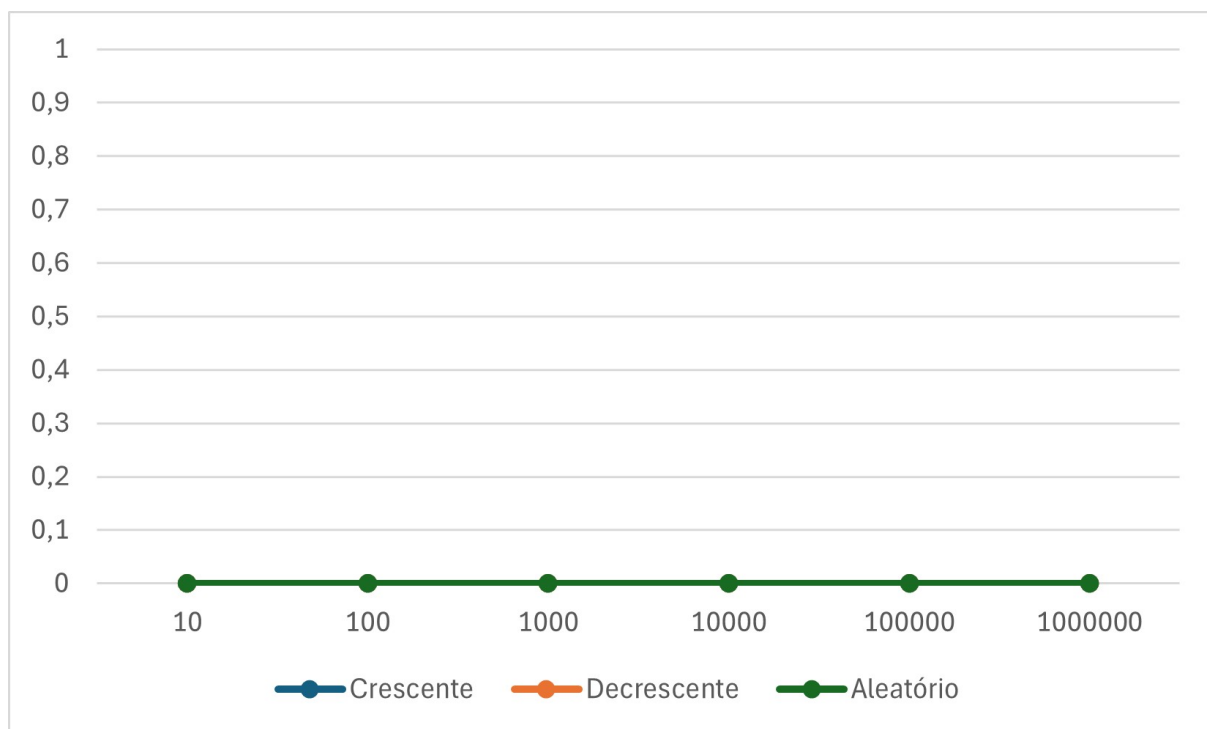


Figura 10: Gráfico de tempo por segundo do algoritmo *Shell Sort*

5.5 Comparação dos algoritmos

	10	100	1000	10000	100000	1000000
Insertion Sort	0,000001	0,000002	0,000013	0,000144	0,001397	0,014323
Selection Sort	0,000266	0,000348	0,002002	0,131247	12,901212	1338,550982
Bubble Sort	0,000319	0,000342	0,002893	0,257234	22,934024	3282,021746
Shell Sort	0,000261	0,000301	0,000367	0,000837	0,008615	0,073479

Figura 11: Tabela de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas crescentes

	10	100	1000	10000	100000	1000000
Insertion Sort	0,000001	0,000025	0,001855	0,203572	17,053548	2114,295511
Selection Sort	0,00023	0,000324	0,001915	0,143501	13,907248	1438,595122
Bubble Sort	0,000407	0,000376	0,001855	0,241055	22,622774	3718,897258
Shell Sort	0,000278	0,000300	0,000407	0,001122	0,010887	0,120733

Figura 12: Tabela de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas decrescentes

	10	100	1000	10000	100000	1000000
Insertion Sort	0,000001	0,000016	0,000916	0,085906	15,17523	1243,076734
Selection Sort	0,000244	0,000328	0,001860	0,132509	12,668543	1339,310597
Bubble Sort	0,000439	0,000701	0,003165	0,242464	32,565813	1243,076734
Shell Sort	0,000260	0,000284	0,000422	0,002766	0,033935	0,421096

Figura 13: Tabela de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas aleatórias

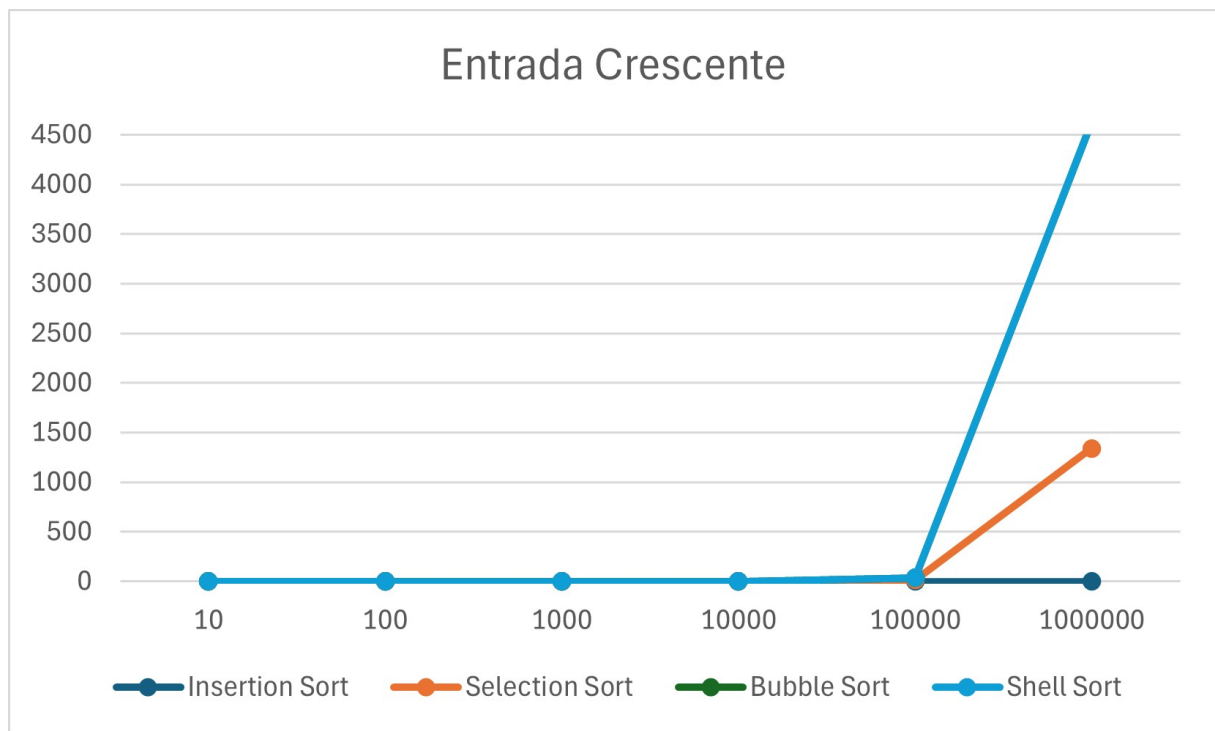


Figura 14: Gráfico de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas crescentes

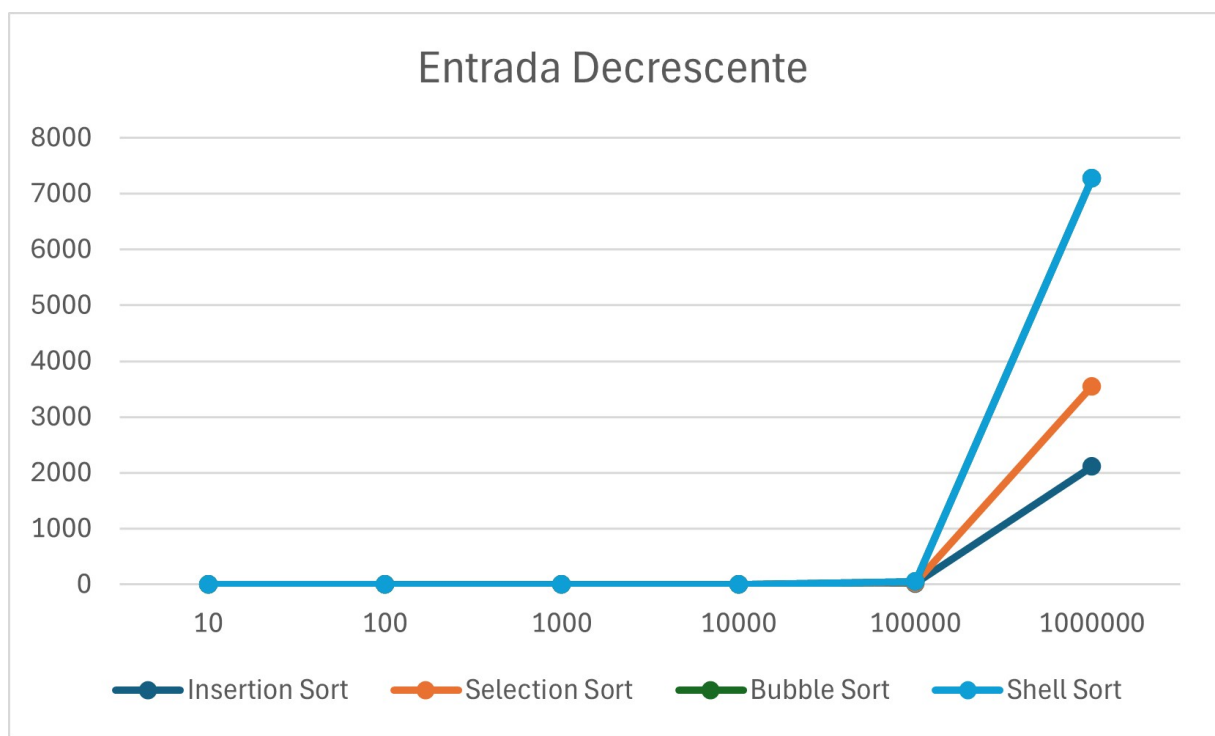


Figura 15: Gráfico de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas decrescentes

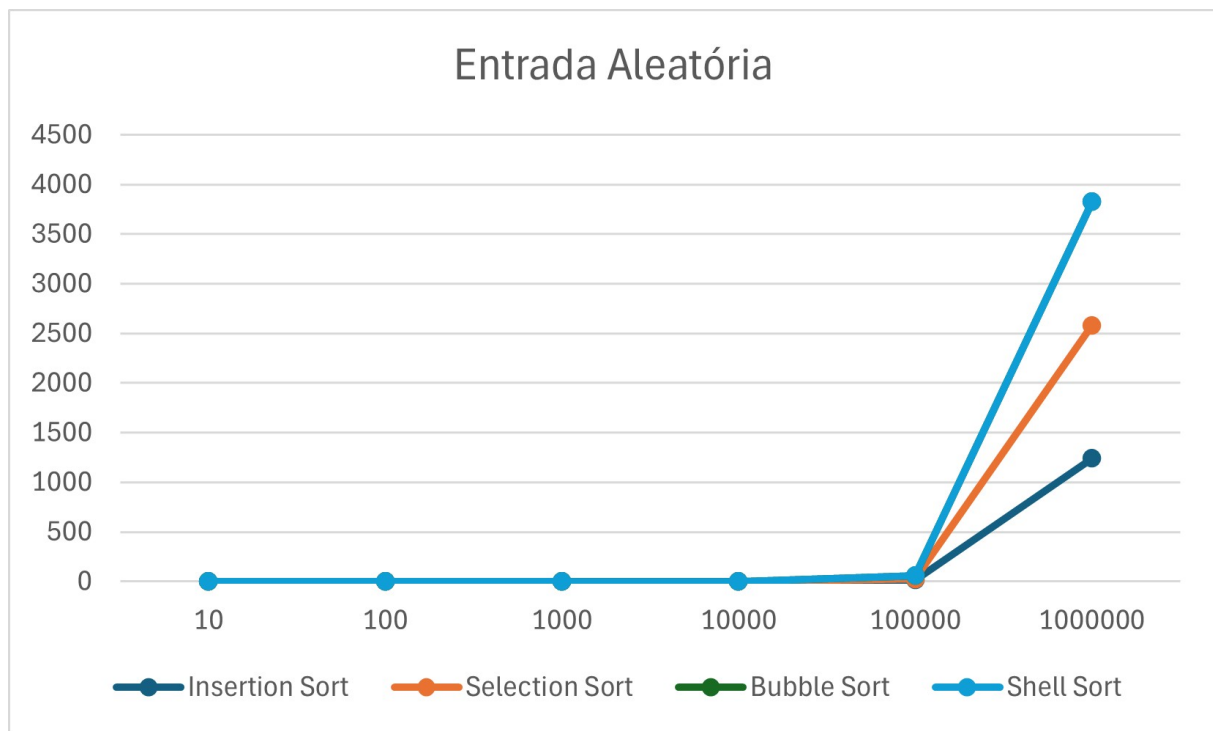


Figura 16: Gráfico de comparação do tempo de execução dos algoritmos *Insertion Sort*, *Selection Sort*, *Bubble Sort* e *Shell Sort* com entradas aleatórias

A partir da análise dos resultados apresentados nas tabelas e gráficos comparativos, observa-se que os algoritmos de ordenação estudados apresentam comportamentos distintos conforme o tipo de entrada. O *Bubble Sort* e o *Selection Sort*, embora simples e didáticos, mostraram-se pouco eficientes em conjuntos maiores, mantendo tempos de execução elevados independentemente da ordenação inicial. Já o *Insertion Sort* demonstrou bom desempenho em listas pequenas ou parcialmente ordenadas, confirmando seu caráter adaptativo.

Por outro lado, o *Shell Sort* destacou-se entre os algoritmos quadráticos por reduzir significativamente o tempo de execução em relação aos demais, especialmente para entradas aleatórias e de maior tamanho. Assim, a comparação evidencia que, embora todos possuam relevância acadêmica para o entendimento dos fundamentos da ordenação, apenas algoritmos mais elaborados, como o *Shell Sort*, oferecem desempenho prático satisfatório em cenários de uso real.

5.6 Merge Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000371	0,000411	0,000522	0,002951	0,014739	0,370827
Decrescente	0,000359	0,000793	0,000760	0,002938	0,030377	0,329605
Aleatório	0,000448	0,000504	0,000827	0,004165	0,040118	0,422724

Figura 17: Tabela de tempo por segundo do algoritmo *Merge Sort*

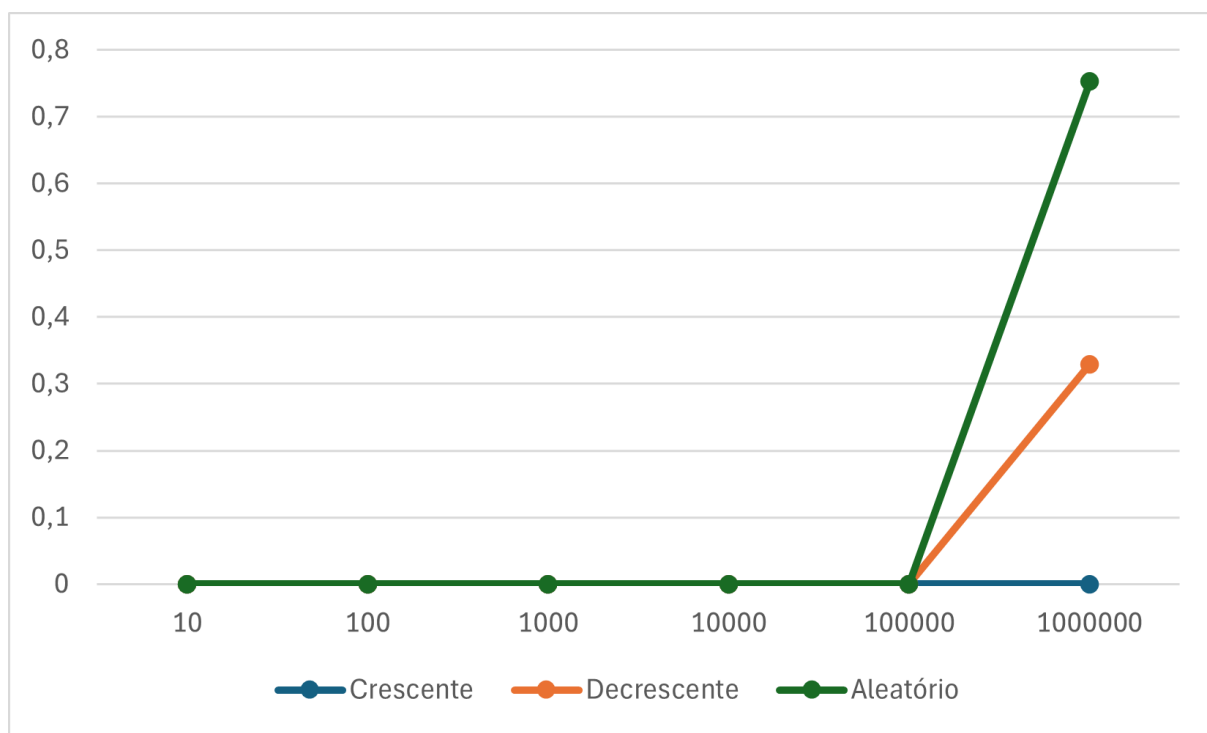


Figura 18: Gráfico de tempo por segundo do algoritmo *Merge Sort*

5.7 Quick Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000004	0,000023	0,001769	0,151146	29,005575	1563,260288
Decrescente	0,000002	0,000026	0,002461	0,159669	16,484379	1426,384129
Aleatório	0,000002	0,000019	0,000187	0,001858	0,023546	0,286666

Figura 19: Tabela de tempo por segundo do algoritmo Quick Sort V1

	10	100	1000	10000	100000	1000000
Crescente	0,000005	0,000005	0,000049	0,257234	0,008845	0,078833
Decrescente	0,000002	0,000010	0,000062	0,000787	0,006438	0,081192
Aleatório	0,000002	0,000014	0,000136	0,003115	0,003115	0,286423

Figura 20: Tabela de tempo por segundo do algoritmo Quick Sort V2

	10	100	1000	10000	100000	1000000
Crescente	0,000003	0,000013	0,002893	0,001391	0,014275	0,206406
Decrescente	0,000003	0,000010	0,000117	0,001043	0,013232	0,222437
Aleatório	0,000003	0,000017	0,000219	0,002213	0,024730	0,292252

Figura 21: Tabela de tempo por segundo do algoritmo Quick Sort V3

Crescente	10	100	1000	10000	100000	1000000
QuickSortv1	0,000004	0,000023	0,001769	0,151146	29,005575	1563,260288
QuickSortv2	0,000005	0,000005	0,000049	0,257234	0,008845	0,078833
QuickSortv3	0,000003	0,000013	0,002893	0,001391	0,014275	0,206406
Decrescente	10	100	1000	10000	100000	1000000
QuickSortv1	0,000002	0,000026	0,002461	0,159669	16,484379	1426,384129
QuickSortv2	0,000002	0,000010	0,000062	0,000787	0,006438	0,081192
QuickSortv3	0,000003	0,000010	0,000117	0,001043	0,013232	0,222437
Aleatório	10	100	1000	10000	100000	1000000
QuickSortv1	0,000002	0,000019	0,000187	0,001858	0,023546	0,286666
QuickSortv2	0,000002	0,000014	0,000136	0,003115	0,003115	0,286423
QuickSortv3	0,000003	0,000017	0,000219	0,002213	0,024730	0,292252

Figura 22: Tabela de comparação dos tempos dos algoritmos Quick Sort V1, V2 e V3

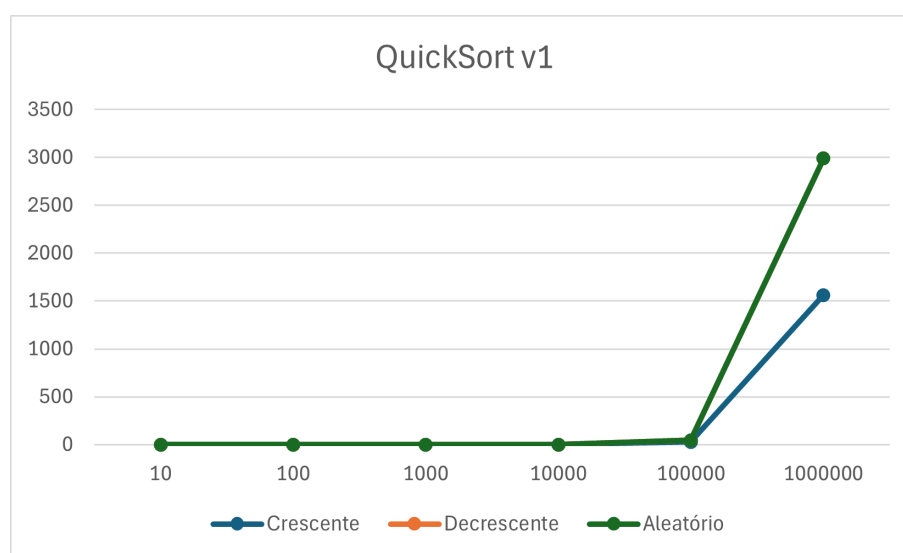


Figura 23: Gráfico de tempo por segundo do algoritmo Quick Sort V1

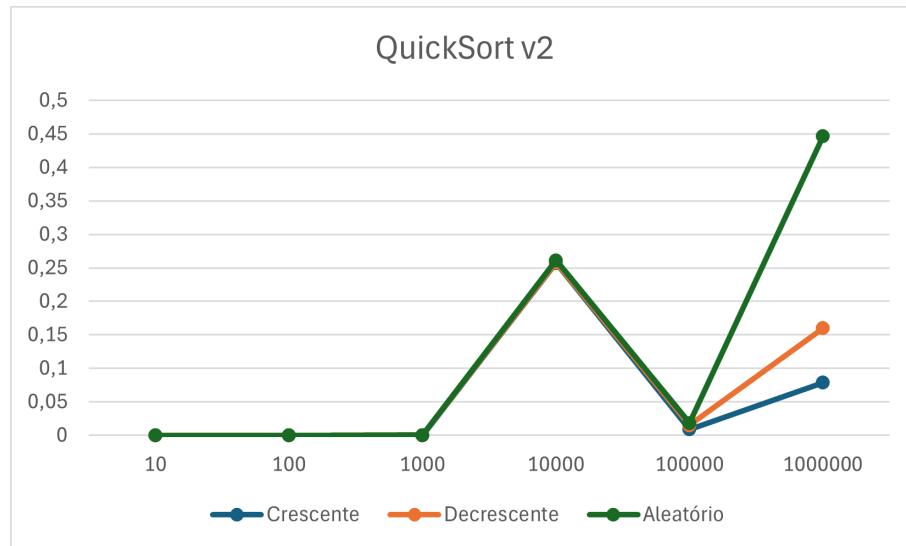


Figura 24: Gráfico de tempo por segundo do algoritmo Quick Sort V2

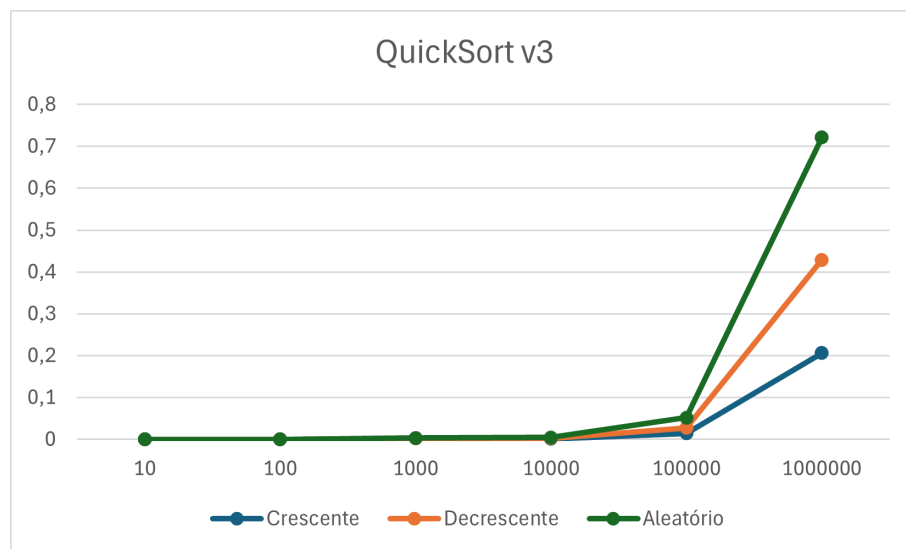


Figura 25: Gráfico de tempo por segundo do algoritmo Quick Sort V3

5.8 Heap Sort

	10	100	1000	10000	100000	1000000
Crescente	0,000004	0,000014	0,000218	0,003014	0,033805	0,427722
Decrescente	0,000002	0,000021	0,000238	0,002879	0,042602	0,404068
Aleatório	0,000002	0,000021	0,000272	0,003170	0,041007	0,507479

Figura 26: Tabela de comparação dos tempos do algoritmo *Heap Sort*

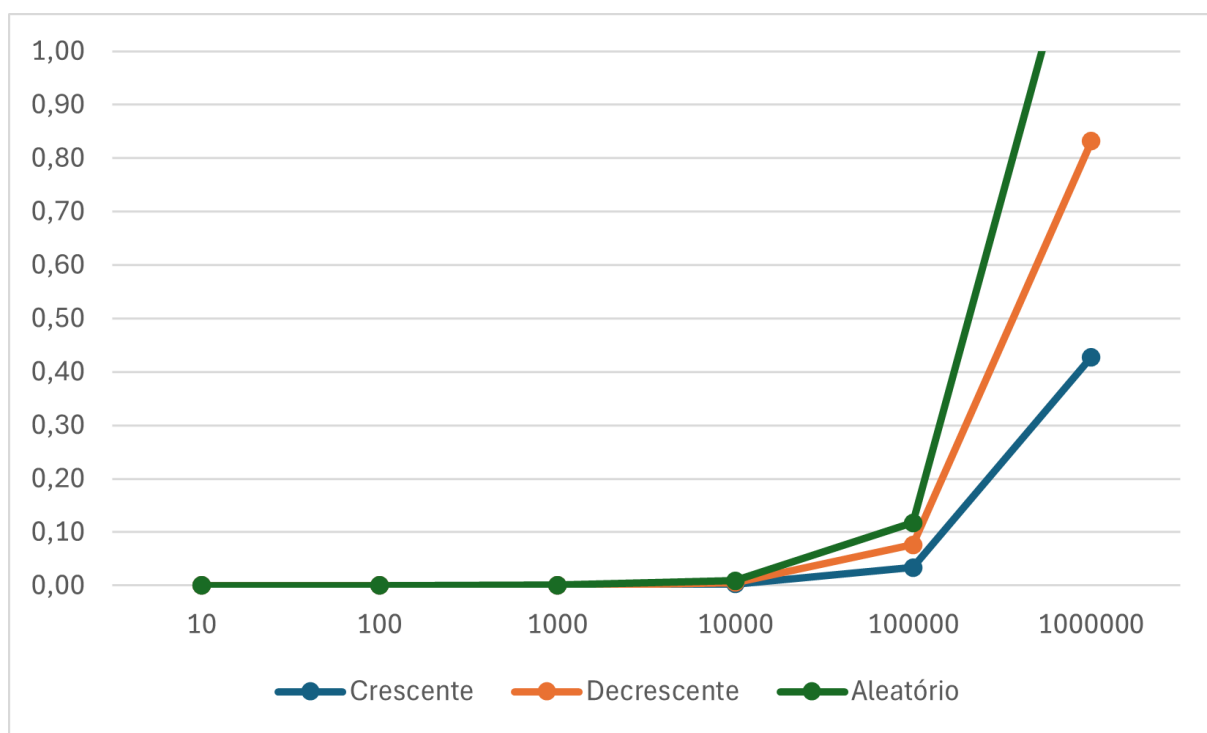


Figura 27: Gráfico de comparação dos tempos do algoritmo *Heap Sort*

6 CONCLUSÃO

Conclusão

A análise comparativa dos algoritmos de ordenação realizada neste trabalho possibilitou compreender de maneira mais ampla o comportamento e a eficiência de diferentes métodos diante de conjuntos de dados de tamanhos variados. Foram executados testes com entradas contendo 10, 100, 1.000, 10.000, 100.000 e 1.000.000 elementos, sob três condições distintas: listas crescente, decrescente e aleatória. Os resultados obtidos demonstraram diferenças significativas de desempenho à medida que o tamanho das instâncias aumentou, evidenciando o impacto da complexidade computacional sobre o tempo de execução.

De modo geral, observou-se que os algoritmos mais simples apresentaram tempos semelhantes nas menores instâncias, mas seu custo de processamento cresceu rapidamente em listas maiores. O *Insertion Sort*, o *Bubble Sort* e o *Selection Sort* apresentaram comportamentos previsíveis, porém ineficientes para grandes volumes de dados devido à sua complexidade quadrática. Já métodos mais estruturados, como o *Shell Sort*, mostraram-se mais estáveis e eficientes, reduzindo consideravelmente o tempo de execução mesmo em entradas aleatórias, nas quais o comportamento dos demais algoritmos foi mais instável.

Entre os métodos basados na técnica de divisão e conquista, tanto o *Merge Sort* quanto o *Quick Sort* destacaram-se por apresentarem desempenho superior nas maiores instâncias testadas. O *Merge Sort*, por empregar um processo de intercalação sistemático e previsível, manteve tempos de execução consistentes e estáveis, independentemente da disposição inicial dos dados, apresentando complexidade $O(n \log n)$ em todos os casos. Já o *Quick Sort*, apesar de também possuir custo médio $O(n \log n)$, demonstrou ser ainda mais eficiente em memória por ser um algoritmo *in-place*, dispensando o uso de estruturas auxiliares. Quando associadas a estratégias adequadas de escolha de pivô — como a mediana de três ou pivôs aleatórios — suas versões mais otimizadas reduziram significativamente a ocorrência do pior caso ($O(n^2)$), mantendo desempenho competitivo mesmo em grandes volumes de dados.

No entanto, entre os algoritmos analisados, o *Heap Sort* apresentou um comportamento particularmente estável e previsível nos experimentos realizados. Por utilizar a estrutura de heap, sua complexidade permaneceu $O(n \log n)$ em todas as instâncias, independentemente da ordem inicial dos dados. Nos testes com 10, 100 e 1.000 elementos, seu desempenho foi semelhante ao do *Merge Sort*, e em algumas execuções superou o *Quick Sort* em estabilidade, especialmente em entradas decrescentes, cenário em que algoritmos baseados em pivôs podem sofrer pequenas variações devido à distribuição dos elementos. À medida que o tamanho das entradas cresceu para 10.000, 100.000 e 1.000.000 elementos, o *Heap Sort* manteve tempos de execução regulares, sem oscilação significativa entre os diferentes tipos de listas, demonstrando robustez frente às características dos dados e confirmando sua capacidade de fornecer ordenações eficientes mesmo em escalas elevadas.

Essas análises reforçam a importância de compreender como o tamanho da entrada e o grau de organização inicial dos dados afetam diretamente o desempenho dos algoritmos. Além disso, demonstram que a escolha adequada do método de ordenação deve levar em conta não apenas sua simplicidade, mas também a natureza do problema, o volume de dados e as restrições de tempo e memória do sistema.

Conclui-se, portanto, que o estudo experimental e teórico dos algoritmos de ordenação é essencial para o desenvolvimento de soluções mais eficientes e escaláveis. A compreensão de como cada técnica reage a diferentes cenários — como entradas ordenadas, inversas ou aleatórias — fornece uma base sólida para a aplicação prática da análise de complexidade e para a construção de sistemas computacionais mais otimizados. Nesse contexto, o *Merge Sort*, o *Quick Sort* e o *Heap Sort* se destacaram como as soluções mais equilibradas e robustas, confirmando sua superioridade em termos de desempenho, previsibilidade e adequação a problemas reais de grande escala.

7 REFERÊNCIAS BIBLIOGRÁFICAS

- [1] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [2] KNUTH, Donald E. *The Art of Computer Programming – Volume 1: Fundamental Algorithms*. 3. ed. Reading: Addison-Wesley, 1997.
- [3] SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2012.
- [4] ZIVIANI, Nivio. *Projeto de Algoritmos: com Implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2011.
- [5] VARELLA, Walter Augusto. *Arquitetura de Solução de Computação em Nuvem*. 1. ed. Rio de Janeiro: Brasport, 2020.
- [6] WIRTH, Niklaus. *Algorithms + Data Structures = Programs*. Englewood Cliffs: Prentice Hall, 1976.
- [7] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- [8] HOARE, Charles Antony Richard. *Quicksort*. *The Computer Journal*, v. 5, n. 1, p. 10–15, 1961.